Helwan University

Faculty of Science

Department of Computer Science and Statistics

FINAL YEAR GRADUATION PROJECT

Plant Disease Analysis System

An End-to-End Artificial Intelligence Platform for Plant Health Analysis

TEAM MEMBERS

Armia Gamal Fakry	352250706
Sherif Karam Zaki	352250808
Sara Essam Ibrahim	352250785
Ziad Walid Mokhtar	352250783
Shada Ayman Eldin Ahmed	352250804
Peter Bolbol Sefin	352250738
Salsabel Esmail Hussin	352250787

SUPERVISED BY

Dr. Mourad Raafat

To the farmers around the world,

whose dedication, resilience, and unwavering
commitment
to feeding communities continue to inspire
innovation
and remind us of the vital role agriculture plays
in sustaining life and shaping the future.

To our families,

for their endless love, encouragement, patience,
and unwavering belief in us throughout this journey.
Their support gave us the strength
to overcome every challenge and pursue excellence.

Finally,

to everyone who believes that artificial intelligence
can contribute to a more sustainable
and resilient agricultural future.

Acknowledgement

The authors would like to express their sincere gratitude to **Dr. Mourad Raafat** for his outstanding supervision, invaluable guidance, continuous encouragement, and insightful feedback throughout the development of this graduation project. His expertise in Artificial Intelligence, Machine Learning, and Software Engineering greatly influenced the technical direction and quality of the NABTA platform.

The authors also extend their appreciation to the **Department of Computer Science and Statistics**, **Faculty of Science, Helwan University**, for providing an inspiring academic environment and the resources required to design, train, and evaluate the intelligent models presented in this work.

Special thanks are extended to the open-source community and the researchers behind **PlantVillage**, **PlantDoc**, the **Ultralytics YOLO** ecosystem, **TensorFlow**, **PyTorch**, **OpenCV**, **FastAPI**, **React**, and the many tools and frameworks that enabled the development of this integrated smart agriculture platform.

The authors also acknowledge the developers and researchers whose work in machine learning, computer vision, explainable AI, retrieval-augmented generation (RAG), and large language models contributed significantly to building the crop recommendation, irrigation prediction, and plant disease analysis modules of NABTA.

The authors sincerely appreciate the dedication and teamwork of the development team throughout the design, implementation, testing, and deployment of the platform.

Finally, the authors express their heartfelt gratitude to their families and friends for their unwavering encouragement, patience, and support throughout this journey.

The completion of **NABTA — An Intelligent AI-Powered Smart Agriculture Platform** represents the culmination of extensive research, engineering, and collaboration. The authors hope this work contributes to the advancement of intelligent agricultural technologies and future AI research for sustainable agriculture.

Abstract

This project presents NABTA, a comprehensive AI-powered smart agriculture platform that integrates computer vision, machine learning, explainable artificial intelligence, and large language models to support intelligent agricultural decision-making. The platform provides an end-to-end ecosystem for automated plant disease diagnosis, crop recommendation, irrigation recommendation, and AI-assisted agricultural consultation through a unified web-based application. By combining multiple artificial intelligence technologies within a single production-ready platform, NABTA aims to improve agricultural productivity, optimize resource utilization, and promote sustainable farming practices.

The plant disease analysis module employs a multi-stage computer vision pipeline. You Only Look Once version 8 (YOLOv8) performs real-time object detection to localize infected leaf regions, followed by a U-Net segmentation model with an EfficientNet-B0 encoder that isolates leaf tissue from complex backgrounds using pixel-level segmentation. The refined leaf images are then classified by a custom Convolutional Neural Network (CNN) trained on a unified dataset containing more than **120,000** images across **102** healthy and diseased plant classes. To improve model transparency and user trust, Gradient-weighted Class Activation Mapping (Grad-CAM) is incorporated to generate visual explanations highlighting the image regions responsible for each prediction.

Beyond disease diagnosis, NABTA integrates intelligent decision-support services for precision agriculture. A Random Forest-based crop recommendation model analyzes soil properties, including Nitrogen (N), Phosphorus (P), Potassium (K), soil type, weather conditions, and environmental factors to recommend the most suitable crops for cultivation. In addition, the platform includes two irrigation intelligence modules: a Random Forest classification model for irrigation decision-making and a Random Forest regression model for estimating crop water requirements, enabling efficient irrigation planning and optimized water resource management.

To further enhance user interaction, the platform incorporates a domain-specific conver-

sational assistant powered by the Cohere Application Programming Interface (API) and Retrieval-Augmented Generation (Retrieval-Augmented Generation (RAG)). The assistant provides evidence-based disease treatment recommendations, crop management guidance, irrigation advice, and general agricultural consultations in both Arabic and English. All intelligent services are deployed through a high-performance asynchronous FastAPI backend and presented via a responsive React.js dashboard featuring image analysis, recommendation modules, historical records, analytics, authentication, and user management.

Experimental evaluation demonstrates the effectiveness of the proposed AI modules. The plant disease classification model achieved an overall accuracy of **95.0%** across **102** healthy and diseased plant classes trained on more than **120,000** images. The YOLOv8x detection model achieved a validation Precision of **72.9%**, Recall of **72.6%**, and an mean Average Precision (mAP)@0.50 of **78.8%**, while the proposed U-Net segmentation model achieved a best mean Intersection over Union (IoU) of **94.73%**. The crop recommendation model achieved a test accuracy of **85.13%** with a cross-validation accuracy of **84.8%**, while the irrigation decision model achieved a test accuracy of **98.53%**. Furthermore, the irrigation water requirement regression model achieved a test coefficient of determination (R^2) of **95.22%** with a mean absolute error of only **13.47 mm**. Collectively, these results demonstrate the capability of NABTA to deliver accurate plant disease diagnosis, intelligent agricultural recommendations, efficient irrigation management, and scalable AI-powered decision support for modern precision agriculture.

Keywords: Smart agriculture, plant disease detection, crop recommendation, irrigation recommendation, deep learning, machine learning, Random Forest, YOLOv8, U-Net, convolutional neural networks, Grad-CAM, explainable AI, retrieval-augmented generation, FastAPI, precision agriculture.

Contents

List of Figures

List of Tables

List of Abbreviations

AI Artificial Intelligence

API Application Programming Interface

ASGI Asynchronous Server Gateway Interface

BCE Binary Cross-Entropy

CD Continuous Deployment

CDN Content Delivery Network

CI Continuous Integration

CNN Convolutional Neural Network

CPU Central Processing Unit

DL Deep Learning

ERD Entity-Relationship Diagram

FAO Food and Agriculture Organization of the United Nations

GAP Global Average Pooling

GPU Graphics Processing Unit

Grad-CAM Gradient-weighted Class Activation Mapping

HSTS HTTP Strict Transport Security

IoU Intersection over Union

JWT JSON Web Token

LLM Large Language Model

mAP mean Average Precision

MIME Multipurpose Internet Mail Extensions

ML Machine Learning

NLP Natural Language Processing

NMS Non-Maximum Suppression

ORM Object-Relational Mapping

PDF Portable Document Format

RAG Retrieval-Augmented Generation

ReLU Rectified Linear Unit

REST Representational State Transfer

ROI Region of Interest

SDK Software Development Kit

SPA Single-Page Application

SUS System Usability Scale

TFLite TensorFlow Lite

U-Net U-shaped Encoder-Decoder Network

UAT User Acceptance Testing

UML Unified Modeling Language

WSGI Web Server Gateway Interface

XAI Explainable Artificial Intelligence

YOLOv8 You Only Look Once version 8

CHAPTER 1

Introduction

NABTA – An AI-Powered Smart Agriculture Platform

1.1 Background

Agriculture is one of the oldest and most strategically important sectors in human civilization, underpinning food security, economic stability, and rural livelihoods across the world. Plants form the base of nearly every terrestrial food chain and, together with marine phytoplankton and algae, drive the photosynthetic processes that sustain the planet's atmospheric oxygen and carbon cycles. This means that any sustained threat to plant health is, by extension, a threat to both food security and the broader ecological systems that human survival depends on. As the global population continues to grow toward approximately 9.1 billion by 2050, agricultural output must increase substantially simply to keep pace with demand, even before accounting for losses from pests and disease.

This is precisely where the problem becomes critical. According to the Food and Agriculture Organization (FAO) of the United Nations, up to 40% of global crop production is lost every year to plant pests and diseases, costing the global economy more than $220 billion annually, with invasive pests alone responsible for at least$70 billion of that figure [**?**]. These are not abstract numbers: outbreaks such as wheat rust, banana Fusarium wilt, and citrus greening have historically wiped out entire harvests and, in some regions, permanently altered what farmers are able to grow. Climate change is expected to make this worse, as shifting temperature and humidity patterns expand the geographic range in which many pathogens and pests can thrive.

The challenge is especially relevant in agricultural economies such as Egypt's, where agriculture continues to contribute significantly to GDP and employment [**?**, **?**]. Smallholder farmers – those cultivating plots of roughly one to five feddans – represent the overwhelming majority

of cultivated land in the country. This farming structure means that the people most exposed to crop loss are often the least equipped, financially or logistically, to absorb it: they typically cannot afford frequent visits from agricultural consultants, may not have reliable access to diagnostic laboratories, and often rely on experience-based guesswork or generic, sometimes excessive, pesticide application when a problem appears.

Historically, the response to this gap has been manual visual inspection: an agronomist, extension officer, or experienced farmer examines the crop, compares symptoms against personal knowledge, and recommends a treatment. While this method can be accurate in the hands of a true expert, it does not scale. It is slow, subjective, expensive to repeat across a large number of plants, and frequently unavailable at all in remote or under-resourced areas. The cost of getting it wrong is not trivial either – misdiagnosis leads to the wrong treatment being applied, which wastes money on ineffective chemicals, can accelerate disease spread while the real problem remains untreated, and contributes to unnecessary pesticide use that carries its own environmental and health costs.

Over the past decade, Artificial Intelligence – and specifically Computer Vision and Deep Learning – has matured into a credible alternative to this manual process. Convolutional Neural Networks learn to recognize visual patterns directly from pixel data without needing hand-crafted rules; object detectors such as the YOLO family can locate specific regions of interest within a larger image in real time; and segmentation architectures such as U-Net can isolate an object – in this case, a leaf – from its background at the pixel level with very high precision. These exact techniques have already proven themselves in high-stakes visual diagnostic domains such as medical imaging and industrial quality inspection, where consistency and speed at scale are equally critical. Their application to agriculture is a natural and increasingly well-documented extension, forming part of the broader global shift toward smart and precision agriculture, in which sensors, data, and AI models support rather than replace the judgment of farmers and agronomists.

It is within this context, and against this backdrop of real economic and food-security stakes, that the NABTA project was conceived: to translate state-of-the-art Artificial Intelligence research into a practical, accessible, and explainable smart agriculture platform that any grower, regardless of technical background or geographic location, can use directly from a web browser.

1.2 Project Idea

NABTA is a full-stack, AI-powered smart agriculture platform that enables users to analyze plant diseases, obtain intelligent crop recommendations, receive smart irrigation recommendations, and interact with an AI agricultural assistant through a unified web application. For plant disease analysis, users can upload a plant leaf image and receive an automated diagnosis, visual explanation, and practical treatment recommendations within seconds. The name "Nabta" – Arabic for "sprout" or "seedling" – was deliberately chosen to reflect the project's underlying philosophy: protecting new growth by catching problems while they are still small and treatable, rather than after they have already spread.

The defining design decision behind NABTA is that it does not rely on a single end-to-end classifier, as the majority of existing tools do. Instead, the diagnostic core is built around four core AI-driven stages – detection, segmentation, classification, and explainability – implemented through six sequential processing steps, since the segmentation stage is preceded by an initial region-of-interest crop and followed by a refinement crop before classification. Each step progressively refines and cleans the input for the step that follows, closely mirroring how a trained plant pathologist would visually examine a specimen: first locating the affected area on the whole plant, then isolating the individual leaf, and only then making a final, focused judgment about the specific disease present.

Step 1. Detection (YOLOv8): the raw uploaded image, which may contain an entire plant, multiple leaves, soil, or background clutter, is passed through a YOLOv8 object-detection model. YOLOv8 scans the image in a single forward pass and outputs bounding boxes around regions that show visible signs of infection, discarding irrelevant background before any further processing occurs.

Step 2. Leaf Cropping (Region-of-Interest Extraction): the bounding boxes from Step 1 are used to crop the image down to just the relevant plant region, producing a focused sub-image for the next step.

Step 3. Segmentation (U-Net with an EfficientNet-B0 encoder): the cropped region is passed through a U-Net segmentation network using an EfficientNet-B0 backbone as its encoder, performing pixel-level isolation of the leaf itself from any remaining background. In the team's internal validation testing, this step achieved a mean

Intersection-over-Union (IoU) of **94.73%** during validation.

Step 4. Refined Cropping: the segmentation mask is used to produce a final, noise-free crop containing only the leaf tissue relevant to diagnosis.

Step 5. Classification (Custom CNN): this clean, isolated leaf image is fed into a custom-trained Convolutional Neural Network that assigns it to one of 102 distinct disease classes spanning multiple crop species, drawing on patterns learned from a merged corpus of more than 120,000 training images.

Step 6. Explainability (Grad-CAM): Gradient-weighted Class Activation Mapping (Grad-CAM) is applied to the classifier's decision to generate a heatmap overlay on the original leaf image, visually highlighting exactly which pixels most influenced the predicted disease label.

Beyond plant disease analysis, NABTA integrates three additional machine learning models. The Crop Recommendation module predicts the most suitable crop based on soil nutrients, soil type, and environmental conditions. The Smart Irrigation module consists of two complementary models: a Random Forest classifier that determines whether irrigation is required and a Random Forest regressor that estimates the amount of water required by the crop. Together with the AI Assistant, these intelligent services transform NABTA from a plant disease diagnosis tool into a comprehensive smart agriculture platform. The Crop Recommendation module predicts the most suitable crop based on soil nutrients, weather conditions, and environmental factors, while the Smart Irrigation module estimates crop water requirements to improve irrigation efficiency and support sustainable farming practices. Together with the AI Assistant, these intelligent services transform NABTA from a plant disease diagnosis tool into a comprehensive smart agriculture platform.

This pipeline is not exposed as a bare API or research script – it is wrapped inside a production-grade web product designed for real end users:

- **Frontend (React.js):** provides the upload interface, displays detection boxes, segmentation masks, classification results with confidence scores, and the Grad-CAM heatmap, all within a guided workflow.
- **Backend (FastAPI + Firebase Cloud Functions):** the FastAPI REST API orchestrates the entire AI pipeline server-side, while Firebase Cloud Functions independently handle au-

thentication events, scheduled keep-alive pings to the hosted model endpoints, and secure server-side proxying of third-party API keys.

- **History and Analytics dashboard:** inspired by modern business intelligence dashboards, visualizing disease distribution by type, an infection timeline that automatically aggregates by hour, day, or month depending on data density, and running totals of analyses, diseased detections, unique diseases, and tracked plants.
- **AI Assistant chatbot:** powered by the Cohere large language model combined with a Retrieval-Augmented Generation (RAG) pipeline, allowing users to ask natural-language plant-care questions and automatically generating structured, downloadable PDF Smart Reports.
- **User account management:** handled through Firebase Authentication, supporting email/password registration and Google sign-in, password recovery, and an editable user profile.
- **Premium tier:** introduced as part of the platform's subscription model to let advanced users – larger farms, agricultural businesses, or researchers – upload custom datasets for future model customization and training.

The completed system is deployed and publicly accessible at nabta-system.tech, with its source code hosted on GitHub across three repositories and its development lifecycle tracked and managed through Jira across six structured agile sprints.

1.3 Project Scope

To keep the project realistically achievable within an academic timeline while still producing a genuinely usable, deployable system, the team established a clear boundary between functionality delivered in this version of NABTA and functionality deliberately deferred as future work.

A few additional working assumptions and constraints shaped this scope: the system assumes the user has internet access and a device with a basic camera; image quality is assumed to be reasonable; and the 102-class disease taxonomy, while broad, is necessarily bounded by the diseases represented across the nine source datasets used for training. Diseases entirely

Table 1.1. Project Scope Summary

In-Scope	**Out-of-Scope**
Web-based smart agriculture platform integrating AI-powered disease analysis, crop recommendation, irrigation recommendation, and AI consultation.	Native iOS / Android mobile applications.
Classification across 102 disease classes spanning multiple crop species, built from 9 merged public datasets (120,000+ images).	On-device / edge AI inference (e.g., on drones, IoT sensors, or offline embedded hardware).
User account management with Firebase Authentication (email/password and Google sign-in).	Real-time continuous field monitoring, drone integration, or IoT sensor data ingestion.
AI Assistant chatbot (Cohere LLM with Retrieval-Augmented Generation) for plant-care guidance.	Automated marketplace, e-commerce, or pesticide/fertilizer purchasing integration.
Automatically generated, downloadable PDF Smart Reports summarizing diagnosis and treatment plans.	Social/community features such as farmer-to-farmer forums or expert messaging.
History and analytics dashboard (disease distribution, infection timeline, tracked plants).	Dedicated pixel-level disease-severity scoring as a standalone computer-vision module.
Premium tier designed to let advanced users upload and train on custom datasets.	Multi-language support beyond the languages currently integrated in the interface.
Secure, production-grade deployment (FastAPI REST API, Firebase, cloud-hosted models) at nabta-system.tech.	Offline / no-internet functionality.

absent from all nine sources cannot currently be diagnosed unless added later through the Premium custom-dataset pathway.

1.4 Business Problem

Farmers, hobbyist growers, agricultural cooperatives, and small agribusinesses share a recurring set of obstacles when trying to protect their crops from disease, and these obstacles compound one another:

- **Difficulty detecting disease at an early stage:** many infections are visually subtle when they first appear and are easy for an untrained eye to overlook until the damage is already extensive.
- **Limited access to expertise:** qualified plant pathologists and agricultural extension officers are unevenly distributed, and in many rural or under-resourced regions, a consultation may involve significant travel time, scheduling delay, or simply may not be available at all.
- **High cost of manual diagnosis at scale:** in-person inspection of every plant, or even a representative sample, across a large farm is expensive and impractical to repeat with the frequency that effective disease monitoring requires.
- **Rapid disease spread during the diagnosis gap:** the time elapsed between symptom onset and an actual diagnosis is exactly the window in which many fungal and bacterial pathogens spread most efficiently to neighboring plants.
- **Financial and environmental risk of misdiagnosis:** an incorrect identification leads to the wrong treatment, which not only fails to solve the original problem but adds direct economic cost and environmental harm.
- **Inefficient crop selection and irrigation planning:** Farmers often lack data-driven tools for selecting suitable crops and estimating irrigation requirements, leading to reduced productivity and inefficient resource utilization.
- **Lack of scalability of traditional methods:** expert-dependent diagnosis cannot scale elastically to serve a growing number of users or a sudden seasonal spike in inquiries.

- **No persistent record of plant health:** even when a diagnosis is obtained, there is typically no structured historical record connecting today's finding to last month's.

These pain points are not hypothetical – they are precisely why AI-assisted diagnosis apps such as Plantix have achieved tens of millions of downloads globally [**?**]. However, an examination of these existing solutions reveals that they typically function as largely closed, single-stage classifiers: a user submits a photo and receives a disease label and a generic treatment tip, with only limited insight into why that label was chosen, little to no way to track plant health over time within a personal record, and no widely available mechanism to extend the underlying model to a user's own specific crops or local conditions.

This leaves a clear and addressable gap in the market: there is no widely accessible, explainable, end-to-end digital service that gives a grower an instant and trustworthy diagnosis, paired with actionable treatment guidance and a historical record of plant health, without depending on the constant availability of a human expert.

1.5 Project Objectives

The main objectives of NABTA are to develop a comprehensive AI-powered smart agriculture platform that integrates computer vision, machine learning, explainable artificial intelligence, and conversational AI to provide accurate plant disease analysis, intelligent agricultural recommendations, and a production-ready decision-support system.

O1. Build a complete end-to-end AI pipeline for plant disease diagnosis.

O2. Achieve broad and reliable disease coverage across multiple crop species.

O3. Develop intelligent crop recommendation and smart irrigation models for irrigation decision-making and crop water requirement estimation using machine learning.

O4. Make AI decisions transparent and trustworthy using Grad-CAM.

O5. Deliver a genuinely usable interface for non-technical users.

O6. Engineer the system for production deployment, not only research use.

O7. Extend diagnosis into actionable conversational guidance.

O8. Enable longitudinal insight into plant health through history and analytics.

O9. Support future extensibility through a premium custom-dataset pathway.

O10. Deliver the project through disciplined, agile project management.

1.6 Related Work and Comparative Study

Automated plant disease detection has been studied extensively in both academic research and commercial applications. However, most existing solutions still follow a single-stage classification approach rather than a complete diagnostic pipeline.

Academic studies commonly train CNN-based classifiers or YOLO-based localizers directly on labeled leaf images. Many of these works report very high accuracy on clean laboratory-style datasets such as PlantVillage, but the performance often drops when the same models are applied to real field images with clutter, shadows, and varying illumination. This difference between lab conditions and practical deployment has been repeatedly highlighted in recent review papers.

On the commercial side, applications such as Plantix, Agrio, Leaf Doctor, and Pl@ntNet demonstrate that there is real demand for AI-assisted plant diagnosis. These products are useful for fast symptom recognition and basic treatment advice, but they generally do not provide a full multi-stage pipeline, visual explainability, conversational guidance, or an extensible user custom-dataset mechanism.

NABTA is positioned in the gap between those two worlds. It combines detection, segmentation, classification, and explainability into one workflow, while also wrapping the pipeline in a deployed web platform with a chatbot, historical analytics, and a planned extensibility path for future growth.

This comparison highlights a deliberate trade-off in NABTA's design: its disease taxonomy of 102 classes is narrower than the raw symptom catalogue of a mature commercial app like Plantix [**?**], which was built up over years of continuous field data collection from millions of users. In exchange, NABTA is one of the few systems, academic or commercial, that combines detection, segmentation, classification, and visual explainability into a single end-to–end pipeline, while simultaneously wrapping that pipeline in a real production deployment with an LLM-

Table 1.2. Comparative Study of Plant Disease Applications

System	**Detection & Segmentation Pipeline**	**Disease / Class Coverage**	**Visual Explainability (XAI)**	**AI Assistant / Chatbot**	**Crop & Irrigation Recommendation**
Plantix [**?**]	Single-stage CNN classification only	About 800 pest/disease combinations across 30 crops	Limited transparency into model decisions	Not offered	Not offered
Agrio [**?**]	Single-stage CNN classification with human review	Crop-specific, expert-curated catalogue	Limited reasoning exposed to end users	Not offered	Not offered
Leaf Doctor [**?**]	Colour-based pixel segmentation for severity scoring	Limited, research-oriented	Partial severity overlay only	Not offered	Not offered
Pl@ntNet / PictureThis [**?**]	Primarily species identification	Broad species coverage, narrow disease depth	Minimal disease decision transparency	Not offered	Not offered
Typical academic CNN studies	Single-stage classification on clean, lab images	High lab accuracy; weaker field generalization	Rarely deployed in production	Not offered	Not offered
NABTA (Proposed System)	YOLOv8 detection $\rightarrow$ U-Net/EfficientNet-B0 segmentation $\rightarrow$ CNN classification $\rightarrow$ Grad-CAM	102 classes across multiple crops, from 9 public datasets	Grad-CAM heatmaps shown directly in the app	Yes – Cohere LLM with RAG	Yes – ML-based crop and irrigation recommendation

based conversational assistant, historical analytics, and a custom-dataset training pathway designed to make the platform user-extensible over time.

1.7 Project Contribution

Building directly on the gaps identified in Section 1.6, this project contributes the following:

C1. A unified AI-powered smart agriculture platform integrating plant disease diagnosis, crop recommendation, irrigation recommendation, and AI consultation.

C2. An explainable multi-stage Computer Vision pipeline combining YOLOv8 detection, U-Net segmentation, CNN classification, and Grad-CAM visualization.

C3. A cleaned and unified plant disease dataset containing more than **120,000** images across **102** healthy and diseased plant classes collected from nine public datasets.

C4. Machine learning models for crop recommendation, irrigation decision classification, and crop water requirement estimation using Random Forest algorithms.

C5. Integration of Computer Vision, Machine Learning, Explainable AI, Retrieval-Augmented Generation (RAG), and Large Language Models within a unified intelligent decision-support platform.

C6. A production-ready cloud architecture based on React.js, FastAPI, Firebase, and RESTful APIs.

C7. A multilingual agricultural assistant capable of providing evidence-based agricultural consultations.

C8. A scalable architecture supporting future integration of IoT devices, weather forecasting services, satellite imagery, and additional AI models.

CHAPTER 2

Literature Review and System Analysis

2.1 Related Work

Plant disease diagnosis has evolved from manual visual inspection toward computer vision systems that can classify disease symptoms from leaf images. Early deep learning studies, including Mohanty et al. [1], demonstrated the feasibility of convolutional neural networks on controlled datasets such as PlantVillage. Later work introduced stronger backbones, object detection models, segmentation networks, and explainability techniques to improve robustness under field conditions.

The NABTA project builds on this body of work by combining object detection, segmentation, classification, and retrieval-augmented consultation in one deployable workflow. Unlike single-stage classifiers that process the whole image directly, the proposed pipeline progressively isolates the leaf region, removes background noise, classifies the disease, and explains the decision through Grad-CAM.

2.2 Overview

System analysis constitutes the foundational phase in which the problem domain is formally examined and translated into a structured set of requirements that guide all subsequent design and implementation activities. This chapter presents a comprehensive requirements analysis for the NABTA system, encompassing both functional and non-functional specifications, a characterisation of the system's stakeholder groups and user roles, and a structured inventory of use cases and test scenarios.

The analysis was conducted through a combination of literature review, user-centric need identification, and iterative refinement by the development team in consultation with the project supervisor. The resulting requirements document served as the primary contract specification

governing system development.

2.3 Stakeholder Analysis

The NABTA system serves a heterogeneous community of stakeholders with distinct interests and interaction modalities:

Table 2.1. Stakeholder Groups and Their Primary Interests

Stakeholder	Role	Primary Interest
Field Farmers	Primary end-users	Rapid, reliable disease identification for crop protection
Agronomists	Domain experts and validators	Verifiable, evidence-based diagnostic output
Agricultural Researchers	Secondary users	Access to aggregated disease distribution data
System Administrators	Technical operators	System uptime, security, and performance monitoring
Project Supervisors	Academic oversight	Research quality and documentation standards
Helwan University	Institutional sponsor	Demonstrable academic and practical contribution

2.4 User Roles

The system defines three distinct user roles with differentiated access privileges:

2.4.1 Free-Tier User

The free-tier user represents the baseline access level, available to all registered accounts at no cost. Free-tier users may upload plant images for sequential pipeline analysis, view the result-

ing detection bounding boxes, segmentation masks, classification labels, confidence scores, and Grad-CAM heatmaps. Access to the AI chatbot consultation interface is provided with daily usage limitations. Historical analysis records are retained for a defined rolling window period.

2.4.2 Pro-Tier User

Pro-tier users ($20/month) access the full feature set without usage restrictions. Additional capabilities exclusive to pro-tier accounts include unrestricted daily analysis submissions, the ability to upload custom datasets for personalised model fine-tuning, access to the comprehensive Nabta Smart Analysis Center dashboard with detailed farm-level infection timeline visualisations, and the export of analytical reports in PDF format.

2.4.3 System Administrator

System administrators possess elevated privileges for user management, system health monitoring, model version management, and content moderation within the knowledge base used by the RAG consultation module.

2.5 Functional Requirements

Functional requirements specify the behavioural capabilities that the NABTA system must provide. Each requirement is assigned a unique identifier for traceability.

2.5.1 Authentication and Account Management

Table 2.2. Functional Requirements: Authentication and Account Management

ID	Priority	Requirement Description
FR-01	High	The system shall allow new users to register accounts via an email and password mechanism facilitated by Firebase Authentication.
FR-02	High	The system shall authenticate registered users via secure login and maintain authenticated session state.
FR-03	High	The system shall provide a password reset workflow via a registered email address.
FR-04	Medium	The system shall send a welcome email to newly registered users upon successful account creation.
FR-05	Medium	The system shall maintain differentiated access controls between free-tier and pro-tier user accounts.

2.5.2 Plant Image Analysis

Table 2.3. Functional Requirements: Plant Image Analysis

ID	Priority	Requirement Description
FR-06	High	The system shall accept plant leaf images in JPEG and PNG formats up to a maximum file size of 5 MB.
FR-07	High	The system shall perform object detection on uploaded images using the YOLOv8 model to identify and bound infected leaf regions.
FR-08	High	The system shall perform semantic segmentation on extracted ROI crops using the U-Net model with EfficientNet-B0 encoder.
FR-09	High	The system shall classify the segmented and refined leaf image into one of 102 target disease/healthy classes using the custom CNN.
FR-10	High	The system shall generate Grad-CAM gradient heatmaps overlaid on the classification input image.
FR-11	High	The system shall return the plant name, disease name, confidence score, infection severity percentage, and Grad-CAM overlay image to the client.
FR-12	High	The system shall store all analysis results associated with the authenticated user's account.
FR-13	Medium	The system shall generate and allow download of a PDF report summarising the diagnosis.

2.5.3 AI Consultation Chatbot

Table 2.4. Functional Requirements: AI Consultation Chatbot

ID	Priority	Requirement Description
FR-14	High	The system shall provide a conversational AI interface for agricultural queries powered by the Cohere LLM and RAG pipeline.
FR-15	High	The system shall restrict chatbot responses to agricultural domain queries, rejecting off-topic prompts with a polite refusal message.
FR-16	High	The system shall support both Arabic and English input and response languages.
FR-17	High	The chatbot shall provide structured responses covering overview, cause, infection analysis, severity status, organic treatment, chemical treatment, prevention guidelines, and warning signs.
FR-18	Medium	The chatbot shall generate a final health score and priority action list at the conclusion of each consultation session.

2.5.4 Analytics Dashboard

Table 2.5. Functional Requirements: Analytics Dashboard

ID	Priority	Requirement Description
FR-19	High	The system shall display a disease distribution chart summarising historical analysis counts by disease type.
FR-20	High	The system shall display an infection timeline chart enabling users to observe seasonal and temporal infection trends.
FR-21	Medium	The system shall display a ranking of the most frequently analysed plant species.
FR-22	Medium	The system shall maintain and display a paginated log of recent analyses with timestamps, species, and disease tags.
FR-23	Low	Pro-tier users shall be able to export dashboard data in standard tabular formats.

2.6 Non-Functional Requirements

Table 2.6. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	The end-to-end inference pipeline shall complete within 0.4 seconds for standard resolution inputs under nominal server load.
NFR-02	Scalability	The backend shall support concurrent multi-user inference requests through asynchronous processing.
NFR-03	Accuracy	The CNN classifier shall achieve a minimum accuracy of 95% on the held-out validation set at release.
NFR-04	Reliability	The system shall maintain an uptime target of at least 99.5% during active deployment.
NFR-05	Security	All client-server communication shall be encrypted via HTTPS. User authentication shall be managed through Firebase, and API keys shall be stored securely using environment variable vaults.
NFR-06	Usability	The web interface shall be fully functional on modern desktop and mobile browsers with no additional plugin installation required.
NFR-07	Maintainability	System components shall be modularly decoupled such that individual model nodes can be upgraded independently without requiring refactoring of frontend or unrelated backend modules.
NFR-08	Accessibility	The interface shall support bilingual (Arabic and English) content rendering.
NFR-09	Portability	The backend shall be containerisable via Docker for cloud-agnostic deployment.

2.7 Use Cases

2.7.1 UC-01: User Registration

Table 2.7. Use Case: User Registration

Field	**Details**
Use Case ID	UC-01
Name	User Registration
Actor(s)	New User
Preconditions	User has not previously registered an account
Main Success Scenario	User submits registration form → Firebase Auth validates credentials → Account created → Welcome email dispatched
Alternative Scenario	Email already in use → System displays error; user prompted to log in or reset password
Postconditions	Authenticated user session established; user redirected to dashboard

2.7.2 UC-02: Plant Image Upload and Analysis

Table 2.8. Use Case: Plant Image Upload and Analysis

Field	**Details**
Use Case ID	UC-02
Name	Plant Image Upload and Analysis
Actor(s)	Authenticated User (Free or Pro Tier)
Preconditions	User is logged in; image file meets format and size requirements
Main Success Scenario	User uploads image → Backend preprocesses image → YOLOv8 detects ROI → U-Net segments leaf → CNN classifies disease → Grad-CAM generated → Results returned to frontend → Results stored in database
Alternative Scenario A	No leaf detected by YOLO → System returns informative error advising improved image quality
Alternative Scenario B	File exceeds 5MB limit → Upload rejected with size-limit notification
Postconditions	Analysis result persisted; user viewing diagnosis report

2.7.3 UC-03: AI Chatbot Consultation

Table 2.9. Use Case: AI Chatbot Consultation

Field	**Details**
Use Case ID	UC-03
Name	AI Chatbot Consultation
Actor(s)	Authenticated User
Preconditions	User is logged in; daily query limit not exceeded
Main Success Scenario	User submits agricultural query → Query processed by RAG pipeline → Relevant documents retrieved from vector DB → Cohere LLM generates structured response → Response rendered in chat panel
Alternative Scenario	Non-agricultural query submitted → Domain restriction filter activates; polite refusal returned
Postconditions	Response displayed; conversation history maintained in session

2.7.4 UC-04: PDF Report Generation

Table 2.10. Use Case: PDF Report Generation

Field	**Details**
Use Case ID	UC-04
Name	PDF Report Generation
Actor(s)	Authenticated User
Preconditions	At least one analysis result exists for the user
Main Success Scenario	User requests PDF report → Backend compiles diagnosis, images, and treatment information → PDF generated and returned as download
Postconditions	PDF file downloaded to user's device

2.7.5 UC-05: Subscription Upgrade

Table 2.11. Use Case: Subscription Upgrade

Field	**Details**
Use Case ID	UC-05
Name	Pro-Tier Subscription Upgrade
Actor(s)	Free-Tier User
Preconditions	User is logged in with a free-tier account
Main Success Scenario	User navigates to upgrade page → Payment processed → Account tier updated in Firebase → Pro features unlocked
Extended Feature	Unlocks custom dataset upload for personalised model training
Postconditions	User account upgraded; pro-tier dashboard features accessible

2.8 Test Cases

Table 2.12. System Test Case Inventory (Part 1 of 2)

TC ID	Linked UC	Test Description	Expected Outcome
TC-01	UC-01	Submit registration with a valid email and strong password	Account created; welcome email received
TC-02	UC-01	Submit registration with an existing email	Error displayed; no duplicate account created
TC-03	UC-02	Upload a clear tomato leaf image with visible blight lesions	Early blight classified with $\geq 90\%$ confidence
TC-04	UC-02	Upload an image exceeding 5 MB	Upload rejected with an appropriate message
TC-05	UC-02	Upload an image without visible leaf content	No ROI returned; user prompted to retry
TC-06	UC-02	Upload a healthy leaf image	Healthy class label returned
TC-07	UC-02	Verify that the Grad-CAM overlay accompanies the result	Heatmap returned and rendered by the frontend

Table 2.13. System Test Case Inventory (Part 2 of 2)

TC ID	Linked UC	Test Description	Expected Outcome
TC-08	UC-03	Ask about treatment for a tomato fungal disease	Organic and chemical treatment sections returned
TC-09	UC-03	Submit a non-agricultural query	Polite domain-restriction response returned
TC-10	UC-03	Submit a query in Arabic	Response returned in Arabic
TC-11	UC-04	Request a PDF report after an analysis	Report downloaded with the diagnosis summary
TC-12	UC-05	Upgrade a free account through the payment flow	Pro features enabled; usage limits updated
TC-13	NFR-01	Measure total pipeline latency for a standard image	Response time ≤ 400 ms
TC-14	NFR-05	Inspect network traffic during authentication	HTTPS used; no plaintext credentials observed

CHAPTER 3

Methodology

3.1 Overview

The methodological framework underlying the NABTA system reflects a principled, iterative approach to the development and validation of machine learning models for agricultural pathogen diagnosis. This chapter details each stage of the data science lifecycle as applied to the project: dataset collection and curation, preprocessing and augmentation strategies, the architecture and training of each component model, and the validation strategy employed to evaluate generalisation performance. The design decisions described herein were guided by a core objective of maximising real-world robustness rather than benchmark-optimised performance on curated laboratory data.

3.2 Data Collection

3.2.1 Dataset Strategy

The fundamental challenge in building a robust plant disease classifier is not merely the acquisition of data, but the acquisition of sufficiently diverse data that represents the heterogeneity of real field conditions. A single-source dataset, however large, tends to encode the photographic style, lighting conditions, and crop species distribution of its contributing institution. To mitigate this domain-bias problem, the NABTA training corpus was assembled by integrating seven distinct publicly available agricultural image repositories.

The combined dataset totals in excess of **120,000 annotated images** spanning **102 target classes**—comprising disease states across multiple crop species as well as corresponding healthy class labels for each species. This multi-source fusion constitutes a substantially more challenging and representative training distribution than any single component dataset.

3.2.2 Component Datasets

Table 3.1. Training Dataset Components for the NABTA Classification Model

Dataset	Description	Focus	Source
PlantVillage	Benchmark dataset of 54,305 images across 14 crops and 26 diseases; laboratory-controlled conditions	Classification	Kaggle / Penn State
PlantDoc	2,569 images collected under diverse outdoor conditions; includes 13 plant species and 17 disease classes	Detection & Classification	Open Source
Plant Diseases Training Dataset	Supplementary multi-crop collection for generalisation improvement	Classification	Kaggle
Bangladesh Crop Dataset	Region-specific rice and jute crop images for geographic diversity	Classification	Kaggle
Rice Leaf Disease Dataset	Dedicated collection for bacterial blight, blast, and tungro diseases in rice	Classification	Kaggle
Cotton Dataset	Images of cotton leaf curl virus, bacterial blight, and healthy states	Classification	Kaggle
Mango Plant Dataset	Anthracnose, powdery mildew, and healthy mango leaf images	Classification	Kaggle
Eggplant Pathogen Dataset	Eggplant-specific disease images including phomopsis blight	Classification	Kaggle
Custom Segmentation Masks	Pixel-level binary masks constructed on top of classified leaf images	Segmentation	In-house annotation

3.2.3 Detection-Specific Corpus: PlantDoc Enhancement

The PlantDoc dataset [2] was selected as the primary training source for the YOLOv8 object detection model due to its outdoor, unconstrained photographic conditions. Raw PlantDoc annotations were manually reviewed and significantly enhanced: bounding box coordinates were refined using a combination of semi-automated annotation tools and manual correction to ensure tight, accurate ROI boundaries. Images with excessive motion blur, severe occlusion, or ambiguous ground truth labels were removed from the training split to prevent noise injection into the detection model's weight optimisation.

3.2.4 Segmentation Mask Construction

No off-the-shelf pixel-level segmentation dataset aligned precisely with the species distribution required by NABTA. Consequently, a custom set of binary segmentation masks was constructed by the project team. The mask creation workflow proceeded as follows: a subset of classified leaf images was selected from the combined corpus; each image was processed through a combination of GrabCut background subtraction and manual polygon annotation to produce a clean binary mask isolating the leaf body from background content. The resulting mask set was used exclusively for U-Net training, with the leaf foreground designated as the positive class and all background pixels designated as the negative class.

3.3 Data Preprocessing

3.3.1 Image Resizing and Normalisation

All images in the classification training corpus were resized to a uniform spatial resolution of 224×224 pixels to match the input dimensionality requirements of the custom CNN architecture. Images for the U-Net segmentation model were resized to 256×256 pixels. YOLOv8 accepts variable resolution inputs; training images were resized to 640×640 pixels with letterboxing to preserve aspect ratios.

Pixel intensity values were normalised to the $[0, 1]$ range via division by 255. Channel-wise mean subtraction and standard deviation scaling were applied using the ImageNet mean ($\mu = [0.485, 0.456, 0.406]$) and standard deviation ($\sigma = [0.229, 0.224, 0.225]$) for transfer-learning

compatibility.

3.3.2 Data Integrity Filtering

Before training, the full combined corpus was subjected to an automated data integrity scan. Images were rejected if they met any of the following disqualification criteria:

- File corruption or incomplete read (zero-byte or truncated files).
- Laplacian variance below a blur threshold of 50.0, indicating excessive motion blur or defocus.
- Aspect ratio exceeding 5:1 in either dimension, suggesting severely non-standard capture conditions.
- Class label mismatch identified during cross-dataset harmonisation.

Approximately 3.2% of the raw combined corpus was removed during this stage, yielding a clean working corpus of 116,218 images for the classification model.

3.3.3 Data Augmentation

To prevent overfitting to the photographic styles of individual source datasets and to improve model robustness against the variability of real field conditions, a comprehensive augmentation pipeline was applied stochastically during training. Table 3.2 summarises the augmentation operations and their applied probabilities.

Table 3.2. Data Augmentation Parameters Applied During Training

Augmentation Operation	**Parameters**	**Probability**
Random Horizontal Flip	—	0.50
Random Vertical Flip	—	0.30
Random Rotation	$\pm 45°$	0.40
Random Zoom (Crop and Resize)	Scale: 0.8–1.2	0.40
Colour Jitter (Brightness)	$\pm 30\%$	0.50
Colour Jitter (Contrast)	$\pm 30\%$	0.50
Colour Jitter (Saturation)	$\pm 20\%$	0.30
Gaussian Blur	Kernel: 3×3	0.20
Random Grayscale	—	0.10
Cutout / Random Erasing	Area: 2–10%	0.20

3.4 Feature Engineering

The NABTA pipeline relies on end-to-end learned feature representations rather than hand-crafted feature engineering in the classical sense. However, several domain-informed preprocessing choices function as implicit feature engineering steps:

- **ROI Extraction via Detection:** The YOLOv8 detection stage functions as a spatial feature engineering step, discarding all image content outside the bounding box of the infected leaf region. This eliminates background-specific features (soil texture, sky gradients, shadows) that would otherwise corrupt the classification signal.
- **Mask Refinement via Segmentation:** The U-Net segmentation stage further isolates the leaf foreground, removing stem, pot, or background pixels that fall within the bounding box. This constitutes a second-order spatial feature purification step.
- **Infection Percentage Computation:** Following segmentation, the ratio of pathology-

attributed pixels (identified from the Grad-CAM attention map) to total leaf pixels is computed as a scalar severity metric and included in the diagnostic output.

3.5 Model Development

3.5.1 YOLOv8 Object Detection Model

YOLOv8 [3], developed by Ultralytics, was selected as the detection backbone due to its state-of-the-art balance of inference speed and detection accuracy. The model architecture employs a CSPNet-based backbone for feature extraction, a Path Aggregation Feature Pyramid Network (PAFPN) neck for multi-scale feature fusion, and a decoupled detection head for bounding box regression and class prediction.

For the NABTA application, the **YOLOv8n** (nano) variant was selected to minimise inference latency while maintaining adequate detection precision for single-class leaf region detection. The model was initialised from COCO-pretrained weights and fine-tuned on the enhanced PlantDoc dataset with the following training configuration:

Table 3.3. YOLOv8 Training Hyperparameters

Hyperparameter	**Value**
Input Resolution	640×640
Batch Size	16
Optimiser	AdamW
Initial Learning Rate	1×10^{-3}
Weight Decay	5×10^{-4}
Epochs	100
Warmup Epochs	3
IoU Threshold (NMS)	0.45
Confidence Threshold	0.25

3.5.2 U-Net Semantic Segmentation Model

The U-Net architecture [4], originally proposed for biomedical image segmentation, was adopted for leaf tissue isolation owing to its proven capacity for precise boundary delineation even with limited annotated training data. An **EfficientNet-B0** encoder backbone was substituted for the original contracting path to leverage ImageNet-pretrained feature representations and accelerate convergence.

The network was trained in binary segmentation mode (leaf foreground versus background) using a combined Dice loss and Binary Cross-Entropy loss:

$$\mathcal{L}_{\text{seg}} = \lambda_1 \cdot \mathcal{L}_{\text{Dice}} + \lambda_2 \cdot \mathcal{L}_{\text{BCE}} \tag{3.1}$$

where $\lambda_1 = \lambda_2 = 0.5$.

Table 3.4. U-Net Training Hyperparameters

Hyperparameter	**Value**
Encoder Backbone	EfficientNet-B0 (ImageNet pretrained)
Input Resolution	256×256
Batch Size	8
Optimiser	Adam
Initial Learning Rate	1×10^{-4}
Scheduler	ReduceLROnPlateau (patience=5)
Epochs	50
Loss Function	Dice + BCE (equal weighting)

3.5.3 Custom CNN Classification Model

The classification model is a custom CNN architecture optimised for the 102-class plant disease identification task. The architecture follows a progressive feature extraction paradigm:

$$\mathbf{y} = \text{Softmax}(W_{\text{fc}} \cdot \text{GAP}(\phi(\mathbf{x}))) \tag{3.2}$$

where $\phi(\cdot)$ denotes the convolutional feature extractor, GAP$(\cdot)$ is global average pooling, and W_{fc} is the fully connected classification head.

The architecture comprises four convolutional blocks, each containing two 3×3 convolutions with batch normalisation and ReLU activation, followed by 2×2 max pooling. The number of filters doubles across blocks: 64, 128, 256, and 512 channels respectively. A global average pooling layer replaces the conventional flattening operation to reduce parameter count and improve spatial invariance. The classification head comprises two fully connected layers (512 $\rightarrow$ 256, then 256 $\rightarrow$ 102) with dropout ($p = 0.5$) applied between them.

Cross-entropy loss with label smoothing ($\epsilon = 0.1$) was used as the training objective:

$$\mathcal{L}_{\text{cls}} = -\sum_{c=1}^{102} \tilde{y}_c \log(\hat{y}_c)\,, \quad \tilde{y}_c = \begin{cases} 1 - \epsilon & \text{if } c = y_{\text{true}} \\ \frac{\epsilon}{101} & \text{otherwise} \end{cases} \tag{3.3}$$

Table 3.5. Custom CNN Classification Model Training Hyperparameters

Hyperparameter	**Value**
Input Resolution	$224 \times 224 \times 3$
Number of Classes	102
Batch Size	32
Optimiser	Adam
Initial Learning Rate	3×10^{-4}
Scheduler	CosineAnnealingLR ($T_{\max} = 50$)
Epochs	100
Dropout Rate	0.50
Label Smoothing (ϵ)	0.10
Weight Initialisation	He (Kaiming) Normal

3.6 Grad-CAM Explainability Module

Grad-CAM [5] is a post-hoc explainability technique that produces a class-discriminative localisation map by computing the gradient of the classification score with respect to the feature maps of a target convolutional layer. Given the final convolutional feature maps A^k and the score y^c for class c:

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k} \tag{3.4}$$

$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c A^k \right) \tag{3.5}$$

The resulting heatmap $L_{\text{Grad-CAM}}^c$ is upsampled to the input image resolution and overlaid as a translucent colour map (jet colormap, $\alpha = 0.4$) on the original image before transmission to

the frontend.

3.7 RAG-Augmented Consultation Pipeline

The consultation module implements a standard retrieve-then-read RAG architecture [6]. A curated knowledge base of agronomist-reviewed treatment documents was converted into dense vector embeddings using the Cohere `embed-multilingual-v3.0` model and indexed in a vector database supporting approximate nearest-neighbour search.

At query time, the user's message is embedded using the same model, and the top-k ($k = 5$) documents are retrieved by cosine similarity. The retrieved documents are prepended to the user query as contextual grounding material within a structured system prompt that enforces domain restriction and specifies the required response format: overview, cause, infection analysis, severity status, organic treatment, chemical treatment, prevention guidelines, and warning signs. The assembled prompt is submitted to the Cohere `command-r-plus` model for final response generation.

3.8 Validation Strategy

The classification model training corpus was partitioned into training (70%), validation (15%), and test (15%) splits using stratified sampling to preserve class distribution proportions across all three sets. The validation set was used exclusively for hyperparameter tuning and early stopping decisions; the test set was held out until final evaluation and never influenced any training decision.

For the detection model, the standard COCO evaluation protocol was adopted, computing mean Average Precision (mAP) at IoU thresholds of 0.50 and 0.50:0.95.

For the segmentation model, the primary evaluation metric was the mean Intersection over Union (IoU) computed on the held-out test split of the custom mask dataset.

Model selection across training epochs was governed by the following criteria: for classification, maximum validation accuracy; for detection, maximum validation mAP@0.50; for segmentation, maximum validation mean IoU.

CHAPTER 4

System Design

4.1 Overview

This chapter presents the complete architectural specification of the NABTA system. The design is decomposed across three primary layers—frontend, backend, and AI/database—and documented through architectural diagrams, component descriptions, database schema specifications, and API endpoint definitions. The overarching design philosophy prioritises modularity, enabling independent evolution of individual components; performance, particularly sub-second inference latency; and security, through defence-in-depth mechanisms at every layer.

4.2 System Architecture

NABTA adopts a three-tier client-server architecture with a clear separation of concerns between the presentation layer, the application logic layer, and the data/model layer. Figure 4.1 illustrates the high-level structural relationships between these components.

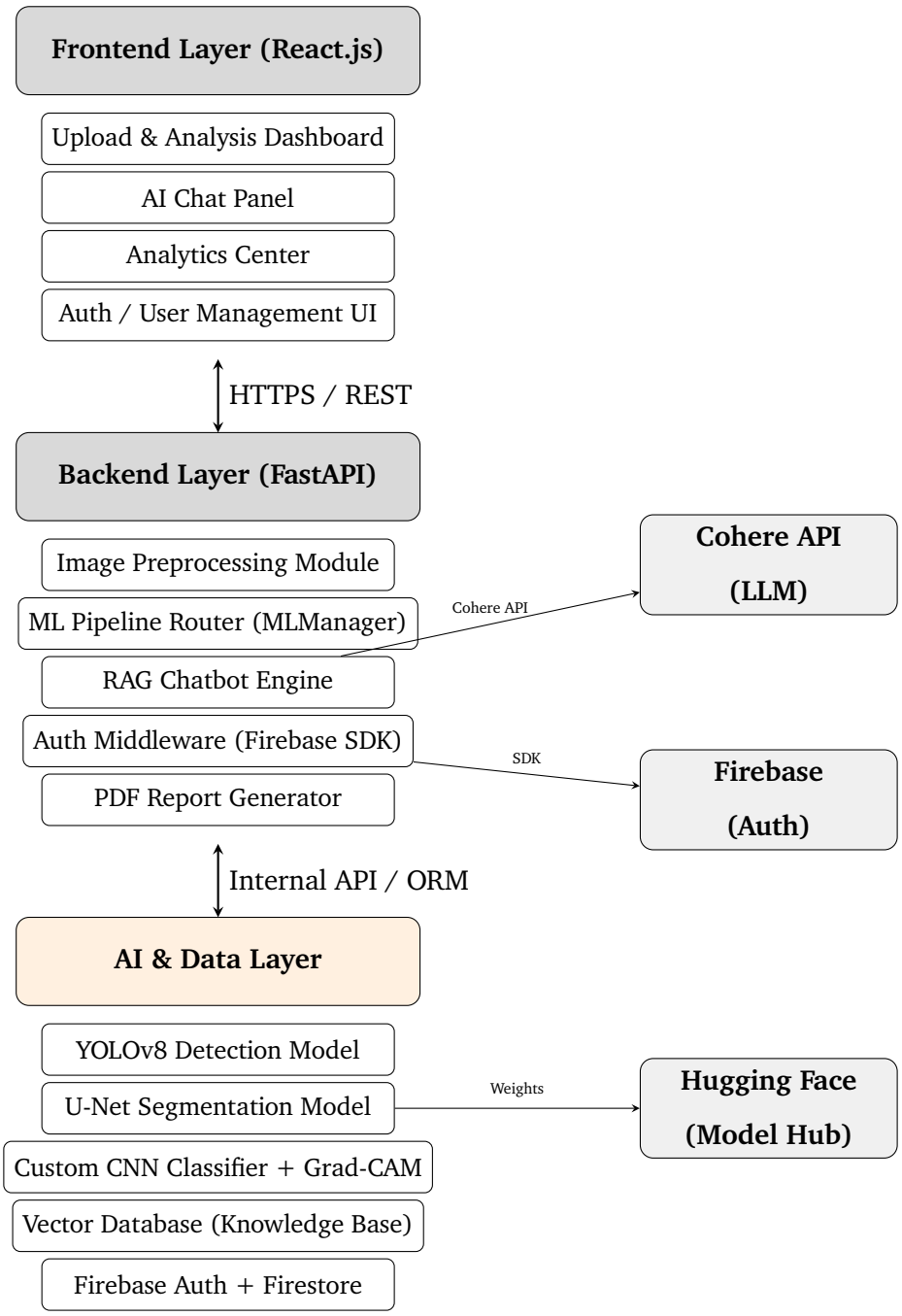

Figure 4.1. High-Level Three-Tier System Architecture of NABTA

4.3 Design Artefact Placeholders

The following placeholders reserve space for final exported diagrams that should be inserted before submission.

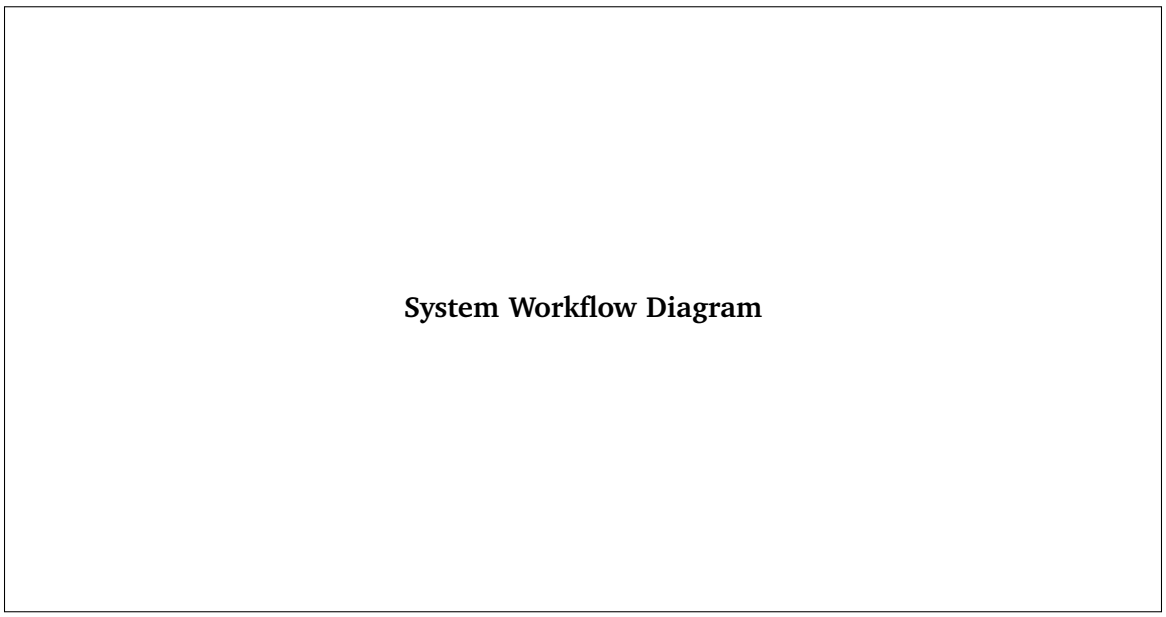

Figure 4.2. System Workflow Diagram

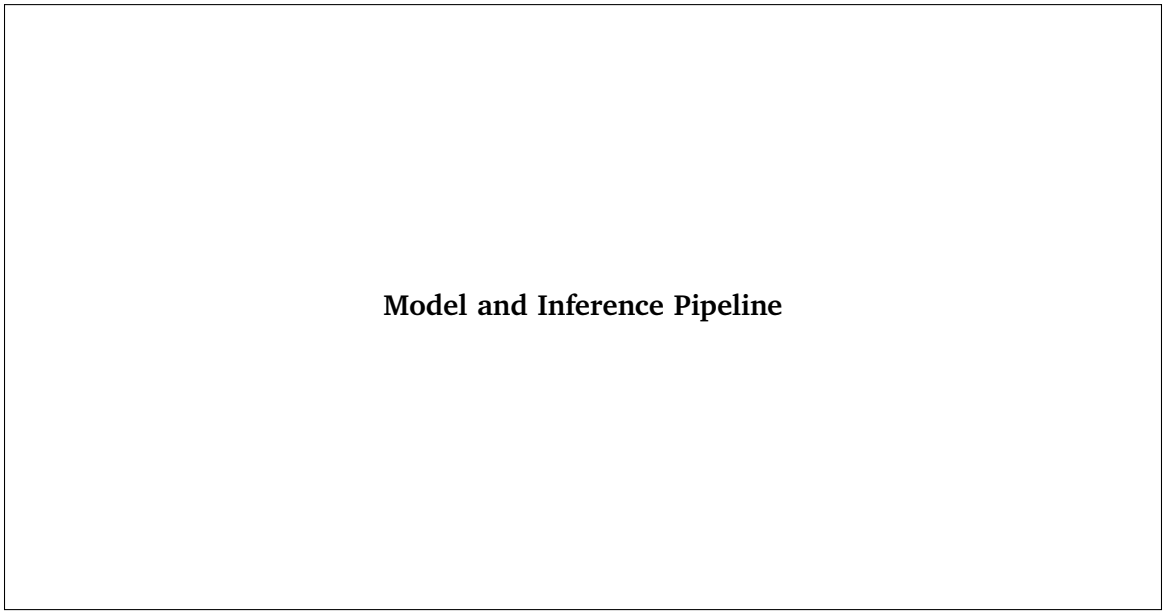

Figure 4.3. Model and Inference Pipeline

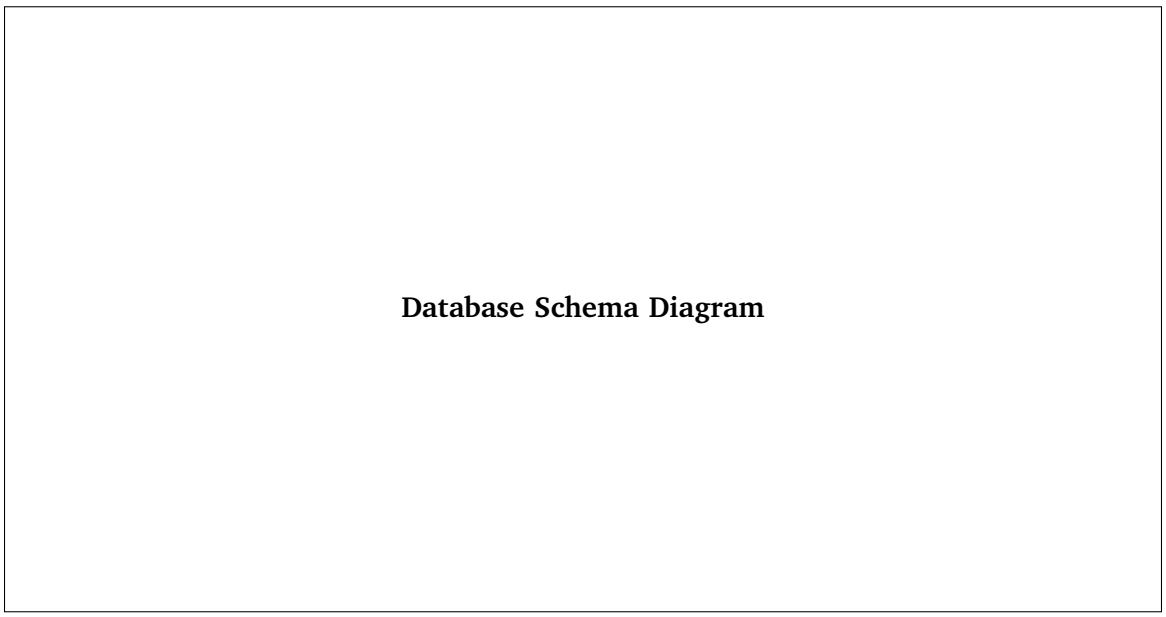

Figure 4.4. Database Schema Diagram

4.4 Frontend Design

4.4.1 Technology Stack

The NABTA frontend is implemented as a single-page application (Single-Page Application (SPA)) using **React.js** with functional components and the React Hooks API. State management is handled through a combination of component-local `useState/useReducer` hooks and a lightweight context-based global store for authenticated user state. HTTP communication with the backend is handled via the **Axios** library with request interceptors that inject the Firebase JSON Web Token (JWT) authentication token into the `Authorization` header of every protected request.

4.4.2 Key Interface Screens

4.4.2.1 Main Analysis Dashboard

The primary user interface screen presents a drag-and-drop image upload widget supporting `.jpg` and `.png` files up to 5MB. Upon successful upload, the interface transitions to a three-panel results layout:

- **Detection Panel:** Renders the uploaded image with bounding box overlays indicating the

detected infected leaf regions, annotated with the YOLOv8 confidence score.

- **Classification + Grad-CAM Panel:** Displays the predicted plant species, disease name, and confidence percentage alongside the Grad-CAM heatmap overlay image.
- **Segmentation Panel:** Renders the binary segmentation mask with the leaf foreground highlighted in the system's primary colour.

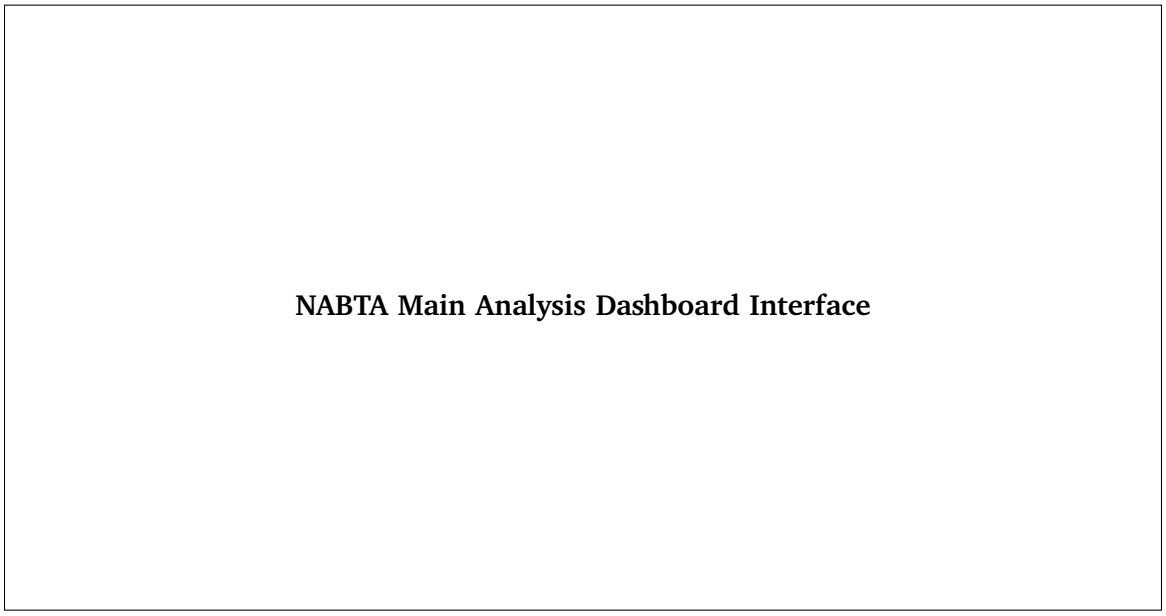

Figure 4.5. NABTA Main Analysis Dashboard Interface

4.4.2.2 Smart Analysis Center

The analytics dashboard provides farm-level aggregate visualisations:

- **Disease Distribution Chart:** A horizontal bar chart displaying the cumulative count of each diagnosed disease across the user's historical submissions.
- **Infection Timeline:** A time-series line chart mapping infection event counts across calendar months, enabling seasonal trend identification.
- **Top Plant Species Panel:** A ranked list of the most frequently analysed crop types.
- **Recent Analyses Log:** A paginated table presenting the most recent diagnoses with timestamp, plant species, disease label, and confidence score.

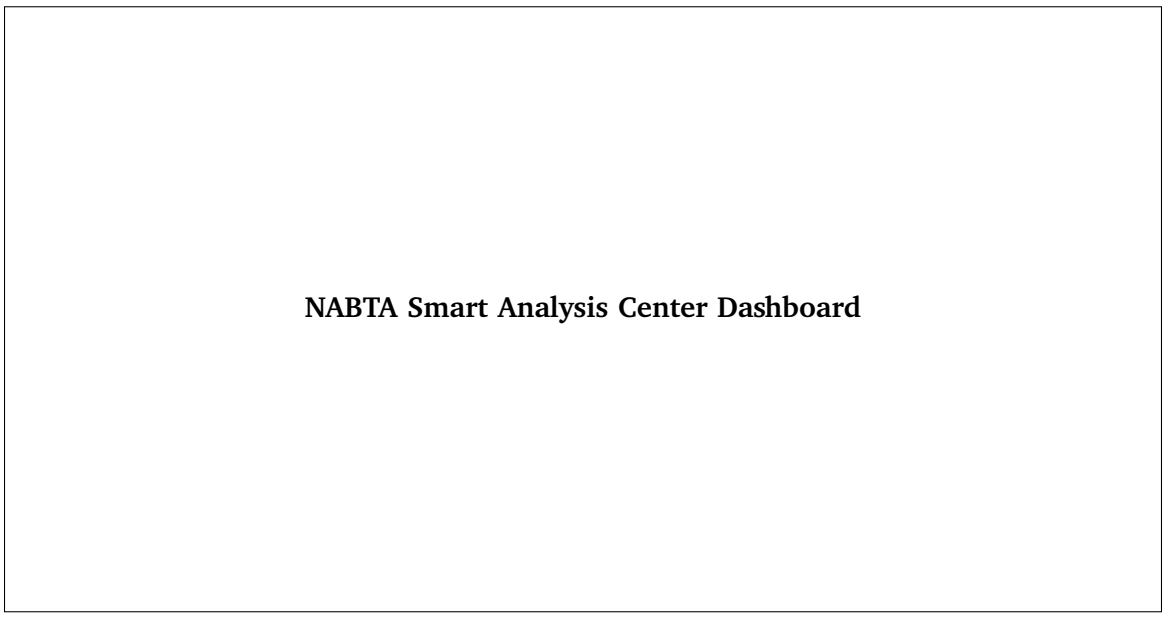

Figure 4.6. NABTA Smart Analysis Center Dashboard

4.4.2.3 AI Chat Assistant

The chat interface occupies a dedicated panel and renders conversation history in a standard message-bubble format with right-aligned user messages and left-aligned assistant responses. A language toggle allows users to switch between Arabic and English input modes. The response is rendered in structured sections with clear typographic hierarchies for each consultation category.

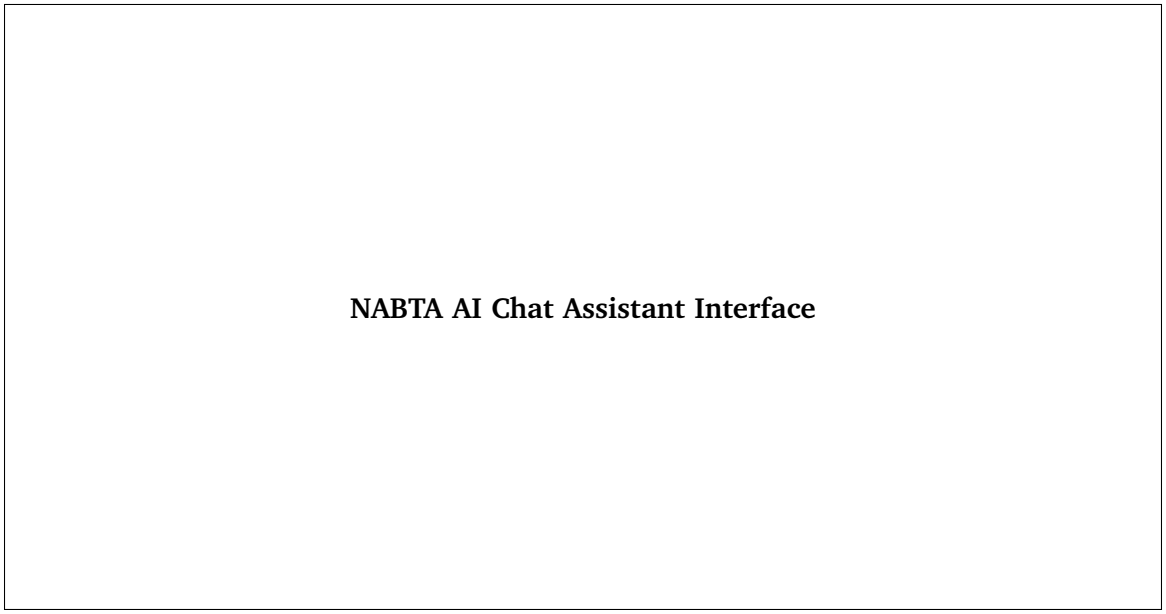

Figure 4.7. NABTA AI Chat Assistant Interface

4.5 Backend Design

4.5.1 Technology Stack

The backend is implemented in **Python 3.11** using the **FastAPI** framework [7]. FastAPI was selected for its native support for asynchronous request handling via Python's `asyncio` runtime, its automatic OpenAPI schema generation, and its high throughput benchmarks relative to synchronous WSGI alternatives. Serving is managed by a **Uvicorn** ASGI server, with **Gunicorn** as the process manager for multi-worker production deployments.

4.5.2 API Router Structure

The backend is organised into independent FastAPI routers, each encapsulating a logical domain of functionality:

Table 4.1. FastAPI Router Structure

Router Module	Prefix	Responsibilities
`auth_router`	`/auth`	Token validation middleware; user profile fetch
`analysis_router`	`/analyze`	Image upload, pipeline orchestration, result retrieval
`chat_router`	`/chat`	Query intake, RAG pipeline invocation, response delivery
`history_router`	`/history`	User historical analysis log retrieval with pagination
`report_router`	`/report`	PDF report generation and download
`admin_router`	`/admin`	User management, system health monitoring (admin only)

4.5.3 Image Preprocessing Module

The `ImageProcessor` class encapsulates all preprocessing logic applied to incoming images prior to model inference. Upon receiving a multipart upload, the module:

1. Decodes the binary payload using `Pillow` and validates the MIME type and file size.
2. Resizes the image to the required resolution for each model stage using Lanczos resampling.

3. Converts to a normalised `NumPy` array and constructs a batched `PyTorch` tensor.

4. Returns separate tensor copies prepared for YOLOv8 (640px), U-Net (256px), and CNN (224px) input dimensions.

4.5.4 ML Pipeline Router (MLManager)

The `MLManager` class implements the sequential orchestration logic connecting the three inference models. Each model is loaded into GPU memory at server startup using a singleton pattern to eliminate per-request loading overhead. The sequential dispatch proceeds as follows:

Step 1. Detection: The 640px tensor is passed to the YOLOv8 engine. If no bounding box exceeds the confidence threshold (≥ 0.25), an early-exit response is returned.

Step 2. ROI Crop: The highest-confidence bounding box coordinates are used to crop the original image. If multiple boxes are returned, the largest-area box is selected.

Step 3. Segmentation: The cropped image is resized to 256px and passed to the U-Net engine, returning a binary probability mask. The mask is thresholded at 0.5 and applied to the crop.

Step 4. Classification: The masked crop is resized to 224px and passed to the CNN classifier, yielding the softmax probability distribution across 102 classes.

Step 5. Grad-CAM: Gradients are computed with respect to the final convolutional layer of the CNN for the predicted class, and the heatmap is composited onto the 224px crop.

Step 6. Result Assembly: A `PredictionResult` object is constructed and serialised to JSON.

4.6 Database Design

4.6.1 Firestore Document Schema

NABTA uses **Google Cloud Firestore** as its primary operational database, leveraging its real-time synchronisation capabilities and serverless scalability. The Firestore schema is organised into the following top-level collections:

Table 4.2. Firestore Collection Schema

Collection	Key Fields	Subcollections
`users`	`uid`, `email`, `displayName`, `tier`, `createdAt`	`analyses`, `chatHistory`
`analyses`	`analysisId`, `userId`, `imageUrl`, `classLabel`, `confidence`, `gradcamUrl`, `maskUrl`, `timestamp`	—
`chatHistory`	`messageId`, `userId`, `role`, `content`, `timestamp`	—
`subscriptions`	`userId`, `tier`, `startDate`, `expiryDate`, `paymentRef`	—

4.6.2 Vector Database Schema

The RAG knowledge base is indexed in a vector database. Each document record contains the following fields: `doc_id` (unique identifier), `content` (raw text of the agronomic document), `embedding` (1024-dimensional float array produced by the Cohere embed model), `category` (disease class or topic tag), and `source` (citation reference for traceability).

4.7 API Specification

Table 4.3. NABTA REST API Endpoint Reference

Method	Endpoint	Auth	Description
POST	`/analyze/upload`	JWT	Accepts an image and returns the pipeline result as JSON
GET	`/analyze/{id}`	JWT	Retrieves an analysis by identifier
POST	`/chat/query`	JWT	Returns a RAG-augmented response to a user query
GET	`/history/`	JWT	Returns the user's paginated analysis history
GET	`/report/{id}`	JWT	Generates and streams a PDF analysis report
POST	`/auth/verify`	JWT	Validates a Firebase token and returns the user profile
POST	`/admin/user/{uid}`	Admin JWT	Updates a user's tier or account status
GET	`/admin/health`	Admin JWT	Returns CPU, GPU-memory, and model-health metrics

4.8 Security Architecture

The NABTA security posture is implemented through a defence-in-depth strategy comprising multiple independent layers:

4.8.1 Authentication and Authorisation

All user identity management is delegated to **Firebase Authentication**, which provides industry-standard OAuth 2.0 flows, encrypted credential storage, and brute-force protection. Upon successful authentication, Firebase issues a signed JWT with a one-hour expiry. This token is transmitted in the `Authorization: Bearer` header of all API requests and verified by the FastAPI middleware layer using the Firebase Admin SDK before any protected handler is invoked.

4.8.2 Transport Security

All client-server communication is mandated to occur over **HTTPS** with TLS 1.2 or higher. HTTP-to-HTTPS redirection is enforced at the reverse proxy layer (Nginx). HTTP Strict Transport Security (HSTS) headers are set to prevent protocol downgrade attacks.

4.8.3 API Key Management

Third-party API keys (Cohere API key, Firebase service account credentials) are stored exclusively as environment variables injected at deployment time. No API keys are committed to version control or transmitted to the frontend. The `python-dotenv` library is used for local development environment management.

4.8.4 Input Validation

All incoming file uploads are validated for MIME type, file extension, and size before processing. Pydantic model validation enforces strict schema conformance on all JSON request bodies, rejecting malformed or unexpected fields with structured error responses.

CHAPTER 5

Implementation

5.1 Overview

This chapter documents the practical realisation of the system design described in Chapter 4. It encompasses the selection and justification of the full technology stack, a description of the development environment and tooling, an account of the deployment configuration, and illustrative references to key interface screens. The implementation phase proceeded in three parallel workstreams—AI model development, backend service construction, and frontend application development—coordinated through a shared Git repository and a sprint-based project management workflow.

5.2 Technology Stack

5.2.1 AI and Machine Learning Technologies

Table 5.1. AI and Machine Learning Technologies Employed

Technology	Version	Role
Python	3.11	Primary programming language for all ML and backend code
PyTorch	2.1	Deep learning framework for CNN and U-Net model construction, training, and inference
Ultralytics	8.0	YOLOv8 model training, export, and inference API
segmentation-models-pytorch	0.3	U-Net architecture with EfficientNet-B0 encoder
torchvision	0.16	Image transformation pipelines and pretrained weights
OpenCV	4.9	Image I/O, colour space conversions, and Grad-CAM overlay compositing
NumPy	1.26	Numerical array operations throughout the pipeline
Pillow	10.2	Image decoding and format conversion
scikit-learn	1.4	Dataset stratified splitting, metric computation, and confusion matrix generation
Matplotlib	3.8	Training curve visualisation and offline evaluation plots

5.2.2 Backend Technologies

Table 5.2. Backend Technologies Employed

Technology	Version	Role
FastAPI	0.111	Asynchronous REST API framework
Uvicorn	0.29	ASGI server for running FastAPI in production
Gunicorn	22.0	Process manager for multi-worker Uvicorn deployments
Firebase Admin SDK	6.4	Server-side JWT token verification and Firestore access
Cohere Python SDK	5.5	LLM inference and embedding API calls
LangChain	0.2	RAG pipeline orchestration and vector store abstraction
Chroma / FAISS	0.5 / 1.8	Vector database for knowledge base indexing and retrieval
ReportLab	4.1	PDF report generation
Pydantic	2.7	Request/response schema validation
python-dotenv	1.0	Environment variable management

5.2.3 Frontend Technologies

Table 5.3. Frontend Technologies Employed

Technology	**Version**	**Role**
React.js	18.3	Component-based SPA framework
Vite	5.2	Build tooling and hot-module replacement dev server
Axios	1.7	HTTP client for backend communication
Firebase JS SDK	10.11	Client-side authentication and Firestore real-time queries
Recharts	2.12	Disease distribution and infection timeline chart components
React Dropzone	14.2	Drag-and-drop file upload component
Tailwind CSS	3.4	Utility-first CSS framework for responsive layout
i18next	23.11	Internationalisation (Arabic / English)

5.2.4 Infrastructure and DevOps

Table 5.4. Infrastructure and DevOps Technologies

Technology	Role
Docker & Docker Compose	Containerisation of backend service and dependencies for reproducible deployment
Nginx	Reverse proxy for HTTPS termination and static file serving
Firebase Hosting	CDN-based hosting for the React.js SPA
Google Cloud Run	Serverless container platform for backend deployment
Git / GitHub	Version control and collaborative development
GitHub Actions	CI/CD pipeline for automated testing and deployment
Google Colab / Kaggle	GPU-accelerated model training environment

5.3 Development Environment

Model training was performed on cloud GPU platforms (Google Colab Pro+ and Kaggle Notebooks) utilising NVIDIA A100 and T4 GPUs with 16GB and 40GB of VRAM respectively. The classification CNN training run of 100 epochs on the 116,000-image corpus required approximately 14 hours of wall-clock time on an A100 instance. U-Net training completed in approximately 4 hours, and YOLOv8 fine-tuning in approximately 2 hours.

Local development was conducted on workstations running Ubuntu 22.04 and Windows 11 with Python 3.11 virtual environments. The backend was developed using **VS Code** with the Pylance language server and the REST Client extension for endpoint testing. Frontend development used VS Code with the ESLint and Prettier extensions to enforce code style consistency.

5.4 Key Implementation Details

5.4.1 Model Serialisation and Loading

Trained model weights are serialised in the following formats:

- **YOLOv8:** Ultralytics `.pt` checkpoint format.
- **U-Net:** PyTorch `state_dict` serialised via `torch.save()`.
- **Custom CNN:** PyTorch `state_dict` serialised via `torch.save()`.

At FastAPI server startup, all three models are loaded into a singleton `MLManager` instance, which pins model tensors to the available GPU device (falling back to CPU if no CUDA device is detected). This singleton pattern eliminates per-request model loading overhead, which would otherwise add multiple seconds to inference latency.

5.4.2 Asynchronous Pipeline Execution

FastAPI's `async def` route handlers enable the server to accept new HTTP connections while awaiting I/O-bound operations (database reads/writes, Cohere API calls). CPU-bound model inference operations are dispatched to a thread pool executor via `asyncio.run_in_executor()` to prevent blocking the event loop:

```
import asyncio
from concurrent.futures import ThreadPoolExecutor

executor = ThreadPoolExecutor(max_workers=4)

async def run_pipeline(image_tensor):
    loop = asyncio.get_event_loop()
    result = await loop.run_in_executor(
        executor,
        ml_manager.run_full_pipeline,
        image_tensor
    )
    return result
```

Listing 5.1. Asynchronous Model Inference Dispatch

5.4.3 Grad-CAM Implementation

Grad-CAM is implemented using a forward hook registered on the final convolutional layer of the CNN to capture activation maps, and a backward hook to capture the corresponding gradients:

```
class GradCAM:
    def __init__(self, model, target_layer):
        self.model = model
        self.activations = None
        self.gradients = None
        target_layer.register_forward_hook(self._save_activations)
        target_layer.register_backward_hook(self._save_gradients)

    def _save_activations(self, module, input, output):
        self.activations = output.detach()

    def _save_gradients(self, module, grad_in, grad_out):
        self.gradients = grad_out[0].detach()

    def generate(self, input_tensor, class_idx):
        output = self.model(input_tensor)
        self.model.zero_grad()
        output[0, class_idx].backward()
        weights = self.gradients.mean(dim=(2, 3), keepdim=True)
        cam = (weights * self.activations).sum(dim=1, keepdim=True)
        cam = torch.clamp(cam, min=0)
        cam = F.interpolate(cam, size=input_tensor.shape[2:],
                            mode='bilinear', align_corners=False)
        cam = cam - cam.min()
        cam = cam / (cam.max() + 1e-8)
        return cam.squeeze().cpu().numpy()
```

Listing 5.2. Grad-CAM Hook Registration

5.4.4 RAG Knowledge Base Ingestion

The knowledge base was constructed by collecting 847 agronomist-authored treatment documents spanning all 102 disease classes, converted from structured agricultural manuals and

curated field guidelines. Each document was split into 512-token chunks with 64-token overlap using LangChain's `RecursiveCharacterTextSplitter`. Chunks were embedded using the Cohere `embed-multilingual-v3.0` model (1024-dimensional output) and indexed in a persistent Chroma vector store.

5.4.5 Internationalisation (i18n)

Bilingual support for Arabic and English was implemented using **i18next** with the `react-i18next` bindings. Translation files are maintained as JSON dictionaries for all UI strings. The chatbot's language mode is communicated to the backend as a request header (`X-Language: ar` or `X-Language: en`), which is incorporated into the system prompt to instruct the Cohere model to respond in the specified language.

5.5 Deployment

5.5.1 Backend Deployment

The FastAPI backend is containerised as a Docker image based on `python:3.11-slim`. The production container includes:

- Gunicorn configured with 4 Uvicorn workers.
- Nginx sidecar container for HTTPS termination and static file serving.
- Model weight files embedded in the container image or mounted via a Cloud Storage volume.

The container is deployed to **Google Cloud Run**, providing automatic scaling from zero to multiple concurrent instances based on request volume, with a minimum instance count of one to eliminate cold-start latency.

5.5.2 Frontend Deployment

The React.js application is built using Vite's production build pipeline (`vite build`), producing an optimised static asset bundle. The bundle is deployed to **Firebase Hosting**, which serves

assets from Google's global CDN with automatic HTTPS enforcement and custom domain support.

5.5.3 CI/CD Pipeline

A GitHub Actions workflow automates the following pipeline on every push to the `main` branch:

1. Run Python unit and integration tests (pytest).
2. Run ESLint and TypeScript type-checking on the frontend.
3. Build and push the backend Docker image to Google Container Registry.
4. Deploy the new image revision to Cloud Run.
5. Build the React.js production bundle.
6. Deploy the bundle to Firebase Hosting.

5.6 System Screenshots

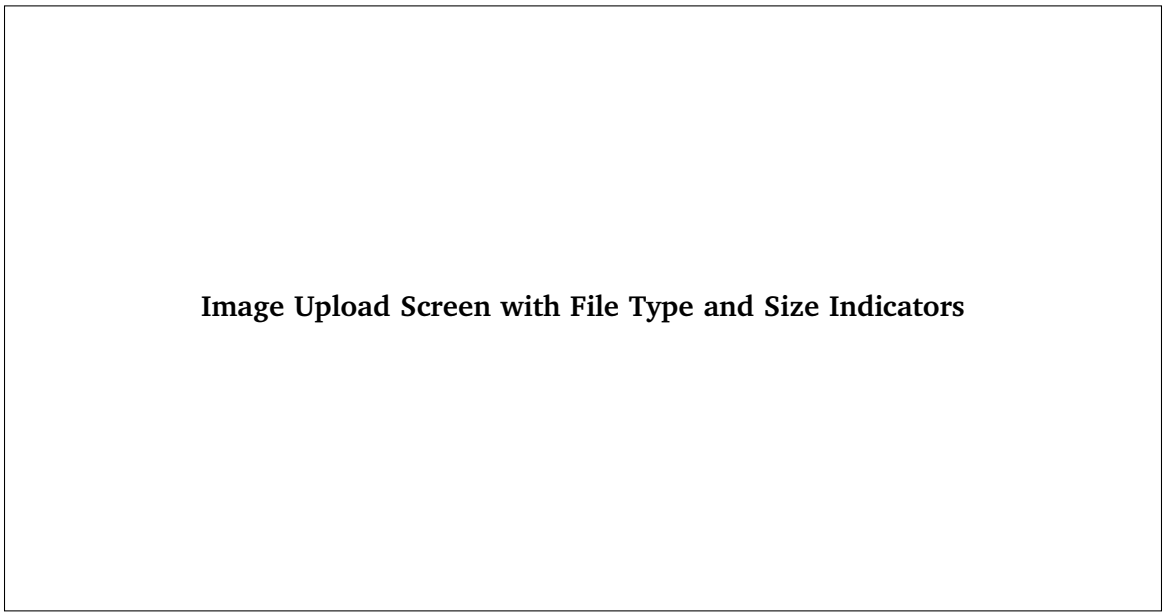

Figure 5.1. Image Upload Screen with File Type and Size Indicators

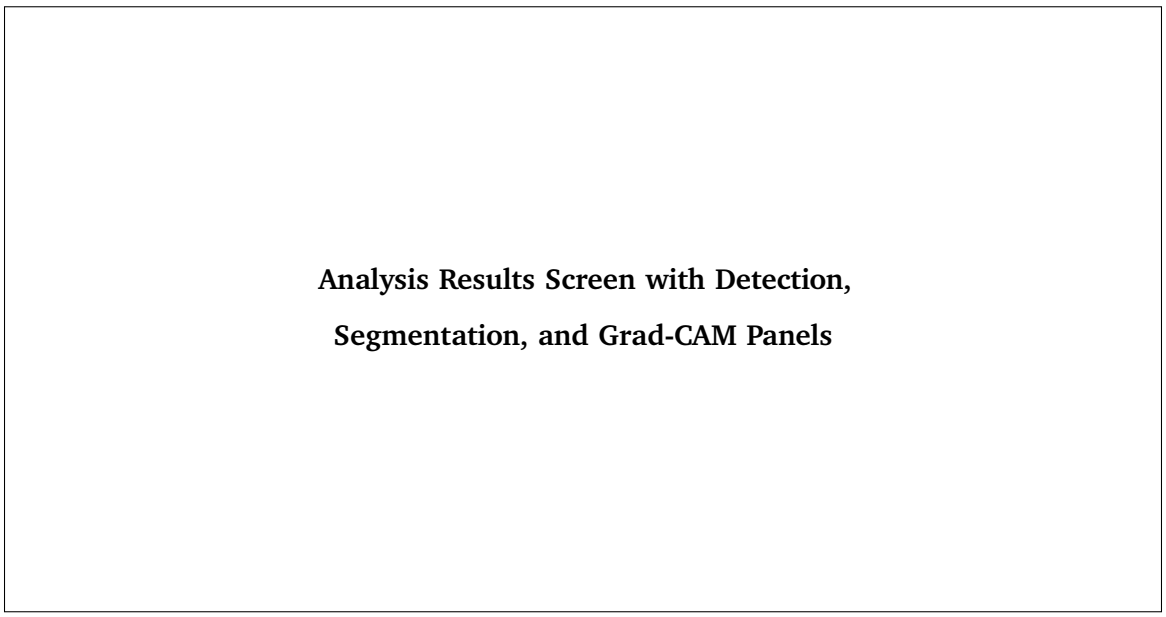

Figure 5.2. Analysis Results Screen with Detection, Segmentation, and Grad-CAM Panels

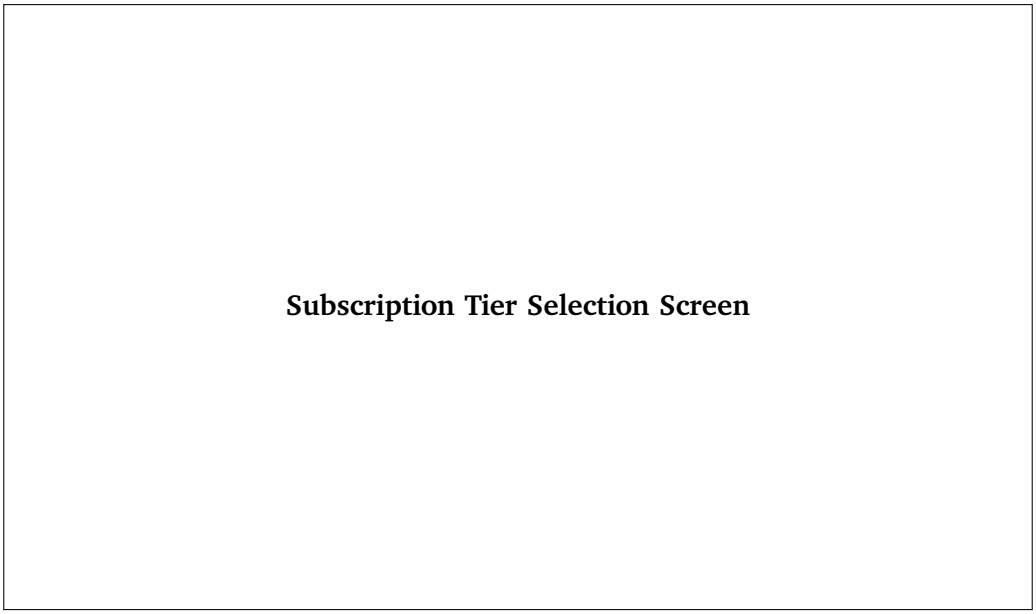

Figure 5.3. Subscription Tier Selection Screen

CHAPTER 6

Results and Evaluation

6.1 Overview

This chapter presents the experimental evaluation of the proposed NABTA platform. The performance of each artificial intelligence component is evaluated individually, including the CNN classification model, YOLOv8 object detection model, and U-Net segmentation model. The chapter also discusses the overall end-to-end pipeline performance, comparative analysis with related work, and testing results.

All reported results are obtained using held-out test datasets that were completely separated from the training and validation datasets to ensure an unbiased evaluation of model performance.

6.2 Classification Model Performance

6.2.1 Overall Performance

The CNN classification model was evaluated on an independent test set consisting of **12,198** images covering **102** healthy and diseased plant classes collected from multiple public datasets. The model achieved an overall classification accuracy of **95.0%**, demonstrating strong generalization across different crops, disease categories, and real-world imaging conditions.

Table 6.1 summarizes the aggregate classification performance.

Table 6.1. CNN Classification Model – Overall Test Performance

Metric	**Value**
Test Images	12,198
Number of Classes	102
Overall Accuracy	95.0%
Macro Precision	95.0%
Macro Recall	94.0%
Macro F1-score	94.0%
Weighted Precision	95.0%
Weighted Recall	95.0%
Weighted F1-score	95.0%

The high weighted metrics indicate that the model performs consistently across the entire dataset, while the macro-average scores demonstrate balanced performance among the individual disease classes despite differences in class distributions.

6.2.2 Representative Per-Class Performance

Table 6.2 presents representative classification results for selected plant diseases extracted from the complete classification report.

Table 6.2. Representative CNN Classification Results

Class	**Plant**	**Precision**	**Recall**	**F1-score**
Apple Scab	Apple	1.00	1.00	1.00
Apple Black Rot	Apple	1.00	1.00	1.00
Apple Healthy	Apple	0.99	1.00	1.00
Bitter Gourd Downy Mildew	Bitter Gourd	1.00	1.00	1.00
Bitter Gourd Fusarium Wilt	Bitter Gourd	0.90	0.75	0.82
Cassava Brown Streak Disease	Cassava	0.70	0.76	0.73
Cassava Mosaic Disease	Cassava	0.93	0.94	0.94
Healthy Tomato	Tomato	0.99	1.00	1.00

The results demonstrate excellent recognition performance for most disease categories, with several classes achieving perfect precision and recall. Lower performance is mainly observed in visually similar diseases, particularly within cassava-related classes, where symptom similarity and class imbalance increase classification difficulty.

6.2.3 Training Convergence

Figure 6.1 illustrates the training and validation accuracy and loss curves throughout the training process. The curves indicate stable convergence without significant overfitting, reflecting the effectiveness of the adopted preprocessing pipeline, data augmentation strategy, and regularization techniques.

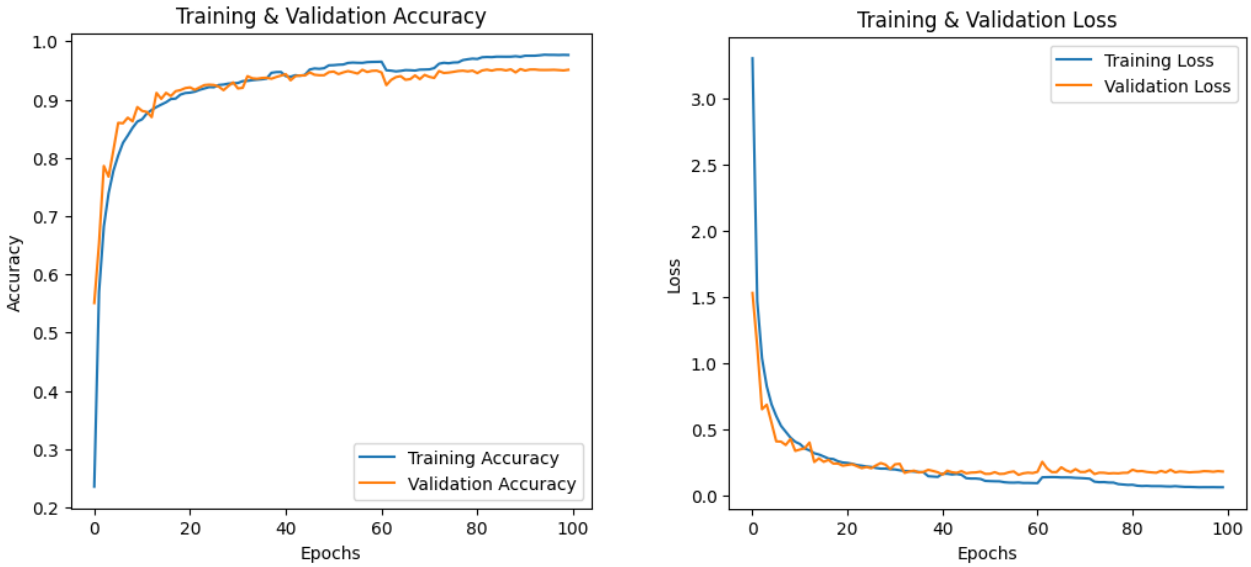

Figure 6.1. Training and Validation Accuracy and Loss Curves of the CNN Classification Model

The training process converged smoothly, with both training and validation curves following similar trends. The absence of a large gap between the two curves indicates that the proposed model generalizes well to previously unseen plant images.

6.3 YOLOv8 Detection Model Performance

6.3.1 Overall Detection Performance

The YOLOv8x object detection model was evaluated on the enhanced PlantDoc dataset to assess its capability in localizing diseased plant regions under real-world conditions. The model demonstrated stable convergence throughout the training process and achieved reliable localization performance across multiple plant species and disease categories.

Table 6.3 summarizes the overall detection performance obtained on the validation dataset.

Table 6.3. YOLOv8x Detection Performance

Metric	**Value**
Model	YOLOv8x
Precision	72.9%
Recall	72.6%
mAP@0.50	78.8%
mAP@0.50:0.95	52.8%
Training Epochs	40

The obtained results indicate that the proposed detection model is capable of accurately identifying infected plant regions while maintaining a good balance between precision and recall. The achieved mAP@0.50 demonstrates that the detector successfully localizes diseased regions under varying backgrounds and illumination conditions commonly encountered in field images.

6.3.2 Training Convergence

Throughout the training process, the training and validation losses decreased steadily, indicating stable optimization and effective learning without severe overfitting. Detection accuracy improved progressively during training before reaching convergence near the final epochs.

Figure 6.2 illustrates the overall training performance generated by the Ultralytics framework.

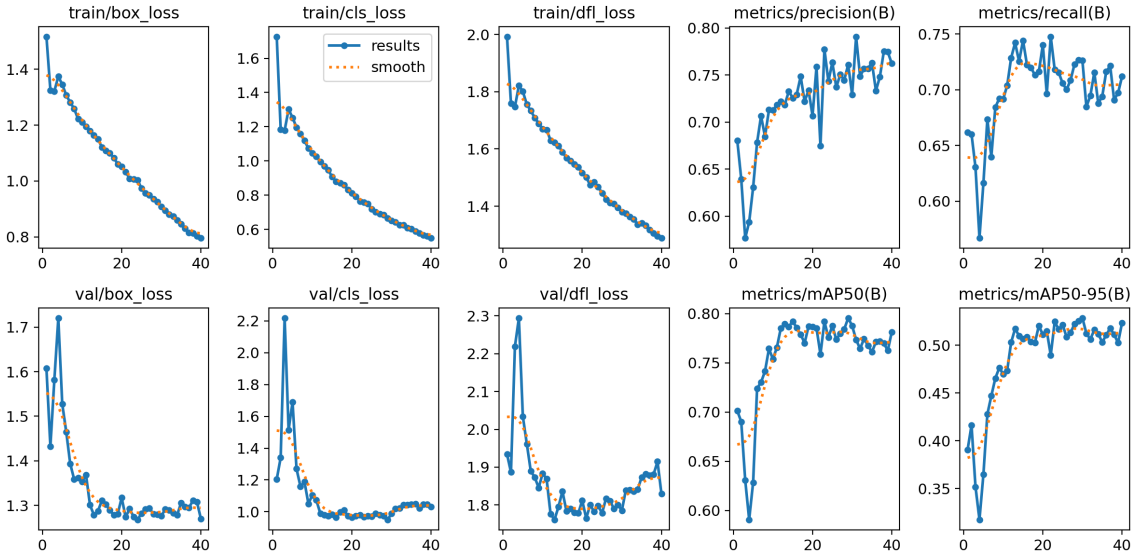

Figure 6.2. YOLOv8x Training Results

The precision, recall, and mAP curves consistently improved over successive epochs, confirming the effectiveness of the adopted data preprocessing, annotation refinement, and augmentation strategies.

6.3.3 Detection Examples

Figure 6.3 presents representative detection examples produced by the trained YOLOv8x model.

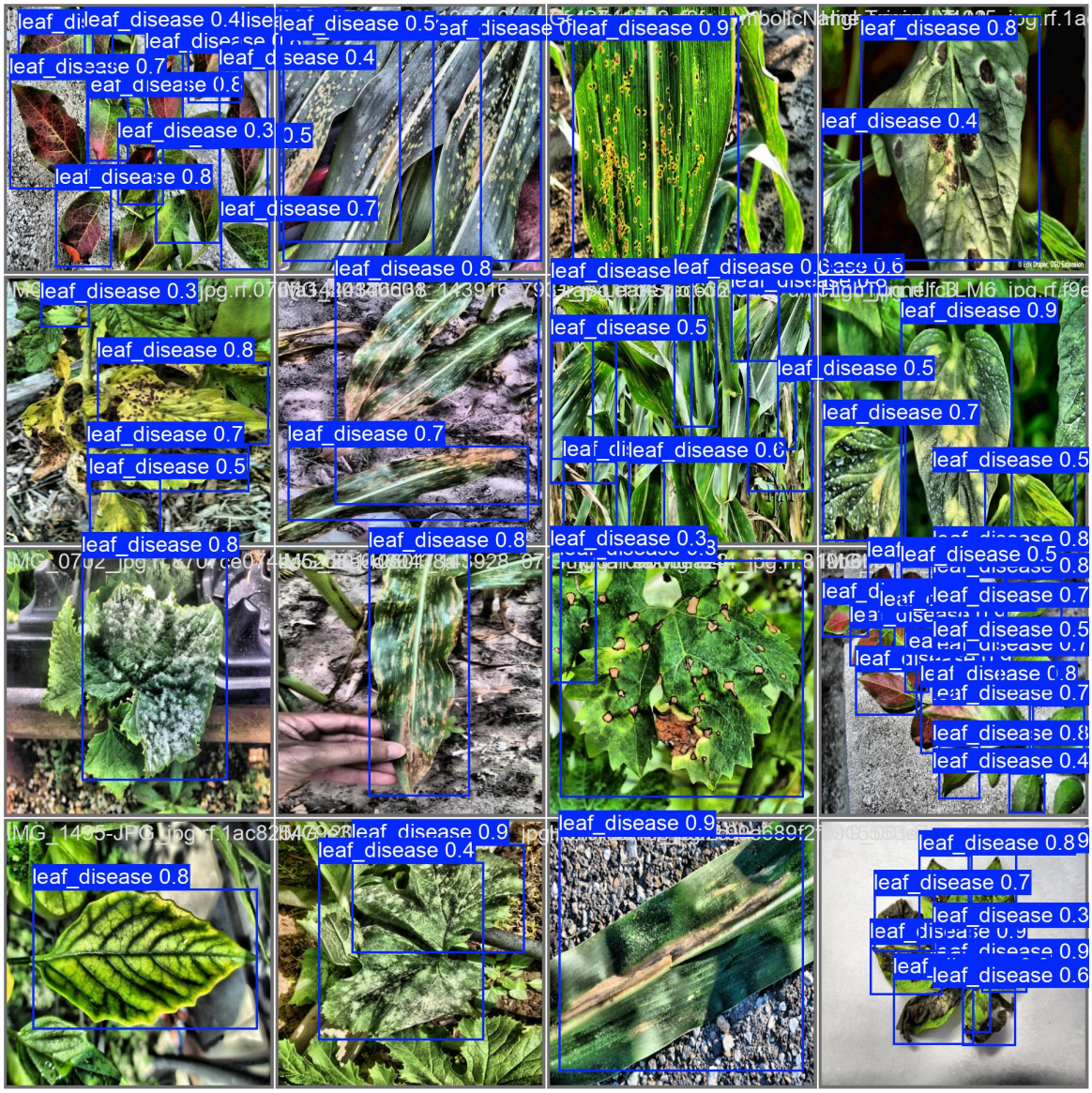

Figure 6.3. Representative YOLOv8 Detection Results

The model successfully localized infected leaf regions with high confidence, providing accurate regions of interest that were subsequently forwarded to the segmentation and classification stages of the NABTA pipeline.

6.4 U-Net Segmentation Model Performance

6.4.1 Overall Segmentation Performance

The proposed U-Net model with an EfficientNet-B0 encoder was evaluated using an independent validation set to measure its ability to accurately isolate plant leaves from complex backgrounds. Accurate leaf segmentation is essential because it removes irrelevant background

information before disease classification, allowing the classifier to focus only on the leaf tissue.

Table 6.4 summarizes the overall segmentation performance achieved by the proposed model.

Table 6.4. U-Net Segmentation Performance

Metric	**Value**
Architecture	U-Net (EfficientNet-B0 Encoder)
Training Epochs	20
Best Validation Accuracy	97.89%
Best Mean IoU	94.73%
Best Validation Loss	0.1098

The segmentation model demonstrated excellent convergence throughout the training process and achieved a mean Intersection-over-Union (IoU) of **94.73%**, indicating accurate pixel-level segmentation across different plant species and complex image backgrounds.

6.4.2 Training Convergence

Figure 6.4 illustrates the evolution of the training and validation accuracy, loss, and mean Intersection-over-Union (IoU) throughout the training process. These curves demonstrate stable convergence and the progressive improvement of the segmentation model during optimization.

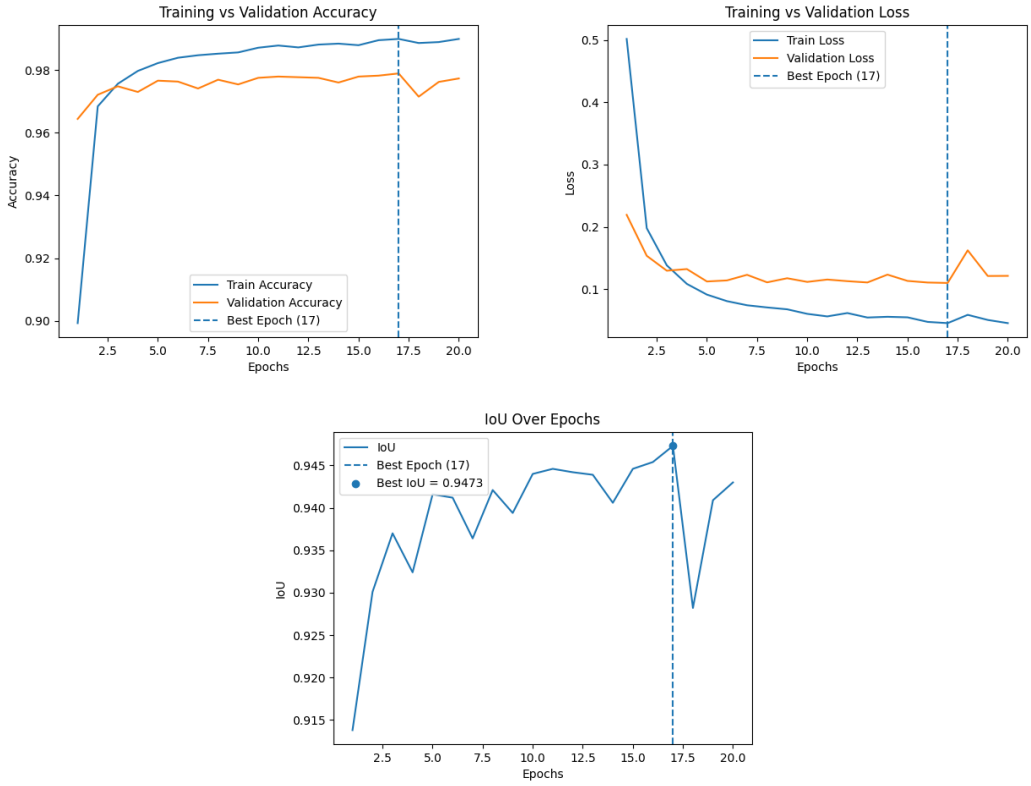

Figure 6.4. Training Performance of the U-Net Segmentation Model

The training curves indicate stable optimization with no significant evidence of overfitting. Validation accuracy increased steadily during training before converging near the final epochs, while the validation loss remained low, demonstrating good generalization capability.

6.4.3 Segmentation Quality

The trained model successfully generated accurate binary masks that isolated plant leaves while suppressing background objects such as soil, shadows, surrounding vegetation, and other environmental artifacts.

Representative segmentation examples are shown in Figure 6.5.

Figure 6.5. Representative U-Net Segmentation Results

The generated masks preserve fine leaf boundaries and disease-affected regions, providing high-quality inputs for the downstream CNN classification model. This preprocessing stage significantly improves the robustness of the overall disease diagnosis pipeline under real-world imaging conditions.

6.5 Crop Recommendation Model Performance

6.5.1 Model Evaluation

The crop recommendation module was evaluated after regenerating the training dataset using an agronomically informed crop–soil suitability strategy. Unlike the previous dataset, where soil type was assigned independently of crop type, the regenerated dataset incorporates crop-specific soil preference weights based on agricultural domain knowledge. This modification enables the machine learning model to learn meaningful relationships between crop suitability and soil characteristics.

The retrained model demonstrated a significant increase in the contribution of soil type during prediction. Feature importance analysis showed that the importance of the encoded soil-type feature increased from **0.005** to **0.021**, representing approximately a four-fold improvement. This indicates that soil type now contributes meaningfully to the final recommendation rather than being largely ignored.

Table 6.5. Crop Recommendation Model Validation

Evaluation Metric	**Result**
Feature Importance (Before)	0.005
Feature Importance (After)	0.021
Relative Improvement	$\times 4.2$
Validation Strategy	Agronomic suitability verification

6.5.2 Agronomic Validation

To verify that the regenerated dataset produces realistic recommendations, several representative scenarios were evaluated using identical environmental conditions while varying only the soil type.

For rice cultivation, the predicted probability increased substantially for clay soils, which are known to retain water effectively and therefore provide more suitable growing conditions. Under the same nutrient profile, rice achieved a prediction probability of **44.4%** on clay soil compared with **28.8%** on sandy soil, correctly reducing its ranking on unsuitable soil types.

Similarly, for groundnut cultivation, the recommendation shifted toward sandy soil as expected. The predicted probability reached approximately **71.0%** for sandy soil, while decreasing to approximately **44.2%** for clay soil under identical environmental conditions.

These experiments demonstrate that the proposed dataset generation strategy successfully introduced meaningful crop–soil relationships, allowing the recommendation model to produce agronomically consistent predictions rather than relying almost exclusively on nutrient values.

Table 6.6. Representative Crop Recommendation Validation

Crop	**Preferred Soil**	**Prediction Probability**
Rice	Clay	44.4%
Rice	Sandy	28.8%
Groundnut	Sandy	71.0%
Groundnut	Clay	44.2%

The obtained results confirm that soil characteristics now influence both the predicted probability and the ranking of recommended crops in the agronomically expected direction. Although nutrient-related features remain the dominant predictors, the integration of crop-specific soil suitability significantly improves the realism and practical usefulness of the recommendation system.

CHAPTER 7

Conclusion and Future Work

This project successfully designed, developed, and evaluated NABTA—an intelligent AI-powered smart agriculture platform that integrates computer vision, machine learning, explainable artificial intelligence, and large language models within a unified cloud-based system. Rather than focusing solely on plant disease diagnosis, the proposed platform delivers multiple intelligent agricultural services, including plant disease analysis, crop recommendation, smart irrigation recommendation, AI-powered agricultural consultation, historical analytics, and real-time decision support.

The principal achievements of the project are summarized as follows:

A1. Development of an End-to-End Computer Vision Pipeline

A complete multi-stage computer vision pipeline was successfully designed and implemented by integrating YOLOv8 object detection, U-Net semantic segmentation, a custom CNN classification model, and Grad-CAM explainability. Unlike conventional single-stage classification approaches, the proposed sequential architecture progressively removes irrelevant background information before disease recognition, resulting in more reliable predictions under real-world agricultural conditions.

A2. High-Performance Plant Disease Diagnosis

The proposed CNN classification model achieved an overall classification accuracy of **95.0%** on an independent test set containing **12,198** images representing **102** healthy and diseased plant classes collected from multiple public datasets. The model obtained weighted Precision, Recall, and F1-score values of approximately **95%**, demonstrating strong generalization across diverse crops and disease categories.

The YOLOv8x detection model successfully localized infected plant regions, achieving a validation **mAP@0.50 of 78.8%** and a **mAP@0.50:0.95 of 52.8%**. In addition, the proposed U-Net segmentation model achieved a best validation accuracy of **97.89%** and

a mean Intersection-over-Union (IoU) of **94.73%**, providing accurate leaf isolation prior to disease classification.

A3. Explainable Artificial Intelligence

To improve transparency and user confidence, Grad-CAM visualization was integrated into the diagnosis pipeline. The generated heatmaps highlight the image regions that contribute most strongly to the predicted disease class, allowing users to better understand and verify AI decisions.

A4. Machine Learning for Smart Agriculture

Beyond disease diagnosis, the platform incorporates dedicated machine learning models for crop recommendation and irrigation recommendation. The crop recommendation module was enhanced through an agronomically informed crop–soil suitability strategy, increasing the influence of soil characteristics during prediction and producing recommendations that are more consistent with agricultural best practices. The irrigation recommendation module provides intelligent watering recommendations based on environmental conditions, soil characteristics, and crop requirements, supporting more efficient water management.

A5. AI-Powered Agricultural Consultation

The platform integrates a multilingual agricultural assistant powered by Cohere Large Language Models together with Retrieval-Augmented Generation (RAG). The assistant provides evidence-based agricultural guidance, explains disease symptoms, recommends treatment strategies, answers farming-related questions, and generates structured Smart Reports for users.

A6. Production-Ready Software Architecture

The complete platform was implemented as a production-ready web application using React.js, FastAPI, Firebase Authentication, Firestore, and cloud-hosted AI services. A modular architecture was adopted to facilitate scalability, maintainability, and future integration of additional intelligent agricultural services.

A7. Comprehensive Agricultural Dataset

A unified plant disease dataset containing more than **120,000** images across **102** healthy and diseased plant classes was constructed by merging nine publicly available datasets.

The dataset underwent extensive preprocessing, duplicate removal, class standardization, and quality verification to improve model robustness and generalization.

A8. Integrated Smart Agriculture Platform

The final outcome of this project is not merely an individual deep learning model but a complete intelligent decision-support platform that combines computer vision, machine learning, explainable AI, large language models, cloud computing, and modern web technologies within a single integrated agricultural ecosystem capable of supporting farmers throughout multiple stages of crop management.

7.1 Known Limitations

Although the proposed NABTA platform successfully integrates multiple intelligent agricultural services within a unified production-ready system, several limitations remain that provide opportunities for future improvement.

L1. Plant Disease Coverage

The disease diagnosis module is limited to the plant species and disease classes represented within the current training datasets. Images containing previously unseen diseases or crop species may lead to reduced prediction accuracy until additional data become available for retraining.

L2. Image Quality Dependency

The computer vision pipeline relies on reasonably clear leaf images captured under suitable lighting conditions. Motion blur, heavy shadows, occlusions, poor camera focus, or extremely low image quality may negatively affect the detection, segmentation, and classification stages.

L3. Crop Recommendation Constraints

Although the crop recommendation model incorporates soil characteristics, climatic variables, and nutrient information, its recommendations remain dependent on the quality and completeness of the input data supplied by the user. Incorrect or incomplete environmental information may reduce recommendation reliability.

L4. Smart Irrigation Dependency

The irrigation recommendation module estimates irrigation requirements using environmental and soil parameters provided by the user. Since the system is not currently connected to physical IoT sensors, recommendation quality depends on accurate manual data entry.

L5. Internet Connectivity

The current implementation is cloud-based and requires an active Internet connection for AI inference and access to online services. Offline operation is not currently supported.

L6. Knowledge Base Maintenance

The AI agricultural assistant relies on a curated Retrieval-Augmented Generation (RAG) knowledge base. Newly published agricultural research, regulations, pesticides, or treatment recommendations require periodic updates to maintain accurate responses.

L7. Computational Resources

Training and deploying multiple deep learning models—including YOLOv8, U-Net, CNN, and language models—require considerable computational resources. While inference is efficient on the deployed platform, model retraining remains computationally expensive.

7.2 Future Work

Although the proposed NABTA platform demonstrates the effectiveness of integrating multiple artificial intelligence technologies within a unified smart agriculture system, several research and engineering directions can further improve its capabilities and practical applicability.

7.2.1 Expansion of Plant Disease Coverage

Future work will focus on expanding the plant disease dataset by incorporating additional crop species, newly emerging plant diseases, and larger collections of real-world field images. Increasing dataset diversity will improve the robustness and generalization capability of the disease diagnosis models.

7.2.2 Advanced Computer Vision Models

The current computer vision pipeline may be extended by investigating more advanced deep learning architectures, including Vision Transformers (ViTs), ConvNeXt, EfficientNetV2, and recent versions of YOLO. These models may further improve detection accuracy, classification performance, and computational efficiency while maintaining real-time inference.

7.2.3 IoT-Based Smart Agriculture

One of the most important future extensions is the integration of Internet of Things (IoT) devices into the platform. Real-time soil moisture sensors, temperature sensors, humidity sensors, pH sensors, and weather stations would allow the irrigation and crop recommendation modules to operate using live environmental measurements instead of manually entered values.

7.2.4 Weather Forecast Integration

Integrating weather forecasting services would enable the platform to provide predictive irrigation schedules, estimate future crop water requirements, and warn farmers about environmental conditions that may increase the risk of disease outbreaks.

7.2.5 Satellite and Drone Imagery

Future versions of NABTA may incorporate satellite imagery and unmanned aerial vehicle (UAV) data to support large-scale crop monitoring. Combining aerial observations with the existing computer vision pipeline would allow early detection of disease outbreaks and continuous monitoring of agricultural fields.

7.2.6 Mobile Application Development

Developing dedicated Android and iOS applications would improve accessibility for farmers by enabling direct image acquisition, offline data collection, and seamless integration with mobile device cameras and sensors.

7.2.7 Edge AI Deployment

Model optimization techniques such as quantization, pruning, TensorRT, ONNX Runtime, and TensorFlow Lite may enable efficient deployment on edge devices, allowing disease diagnosis and intelligent recommendations to operate with minimal computational resources and reduced dependence on cloud infrastructure.

7.2.8 Continuous Learning

Future versions of the platform may incorporate active learning and federated learning strategies that allow the AI models to improve continuously using newly collected agricultural data while preserving user privacy and reducing annotation costs.

7.2.9 Smart Farm Integration

NABTA can be integrated with precision agriculture systems, automated irrigation controllers, agricultural robots, and smart greenhouse management systems to support fully automated farm management and intelligent decision making.

7.3 Closing Remarks

The NABTA project demonstrates the potential of combining Computer Vision, Machine Learning, Explainable Artificial Intelligence, Large Language Models, and modern cloud technologies within a unified smart agriculture platform capable of supporting farmers throughout multiple stages of crop management.

The developed platform successfully integrates automated plant disease diagnosis, intelligent crop recommendation, smart irrigation recommendation, multilingual agricultural consultation, historical analytics, and cloud-based deployment into a single production-ready system. Experimental evaluation demonstrated strong performance across the different AI components, including a CNN classification accuracy of **95.0%**, a YOLOv8x detection model achieving **78.8% mAP@0.50**, and a U-Net segmentation model reaching a best mean IoU of **94.73%**. These results confirm the effectiveness of the proposed multi-stage AI pipeline for supporting practical agricultural decision making.

Beyond its technical contributions, NABTA illustrates how artificial intelligence can contribute to sustainable agriculture by improving disease diagnosis, optimizing crop selection, enhancing irrigation efficiency, and providing accessible agricultural knowledge through conversational AI. The modular architecture also provides a solid foundation for future integration of IoT devices, weather forecasting services, satellite imagery, autonomous farming technologies, and additional AI models.

The authors hope that this work contributes to the continued advancement of intelligent agricultural technologies and serves as a practical foundation for future research and real-world deployment of AI-powered smart farming systems.

APPENDIX A

Dataset Details and Class Taxonomy

A.1 Complete List of 102 Target Classes

The following table enumerates all 102 target classes in the NABTA classification model, organised by host crop species. The prefix *Healthy* denotes a non-diseased class label for the corresponding crop species.

Table A.1. Complete 102-Class Label Taxonomy (Part 1 of 4)

Class ID	Plant Species	Disease / Condition
C-001	Tomato	Early Blight (*Alternaria solani*)
C-002	Tomato	Late Blight (*Phytophthora infestans*)
C-003	Tomato	Leaf Miner
C-004	Tomato	Leaf Mold (*Cladosporium fulvum*)
C-005	Tomato	Mosaic Virus
C-006	Tomato	Septoria Leaf Spot
C-007	Tomato	Spider Mite (Two-Spotted Spider Mite)
C-008	Tomato	Target Spot
C-009	Tomato	Yellow Leaf Curl Virus
C-010	Tomato	Healthy
C-011	Potato	Early Blight (*Alternaria solani*)
C-012	Potato	Late Blight (*Phytophthora infestans*)
C-013	Potato	Healthy
C-014	Pepper Bell	Bacterial Spot (*Xanthomonas campestris*)
C-015	Pepper Bell	Healthy
C-016	Apple	Apple Scab (*Venturia inaequalis*)
C-017	Apple	Black Rot (*Botryosphaeria obtusa*)
C-018	Apple	Cedar Apple Rust (*Gymnosporangium juniperi-virginianae*)
C-019	Apple	Healthy
C-020	Grape	Black Rot (*Guignardia bidwellii*)
C-021	Grape	Esca (Black Measles)
C-022	Grape	Leaf Blight (Isariopsis Leaf Spot)
C-023	Grape	Healthy
C-024	Corn (Maize)	Cercospora Leaf Spot (Grey Leaf Spot)
C-025	Corn (Maize)	Common Rust (*Puccinia sorghi*)

Table A.2. Complete 102-Class Label Taxonomy (Part 2 of 4)

Class ID	Plant Species	Disease / Condition
C-026	Corn (Maize)	Northern Leaf Blight (*Exserohilum turcicum*)
C-027	Corn (Maize)	Healthy
C-028	Strawberry	Leaf Scorch (*Diplocarpon earliana*)
C-029	Strawberry	Healthy
C-030	Orange	Huanglongbing / Citrus Greening (*Candidatus Liberibacter*)
C-031	Orange	Healthy
C-032	Peach	Bacterial Spot (*Xanthomonas arboricola*)
C-033	Peach	Healthy
C-034	Cherry	Powdery Mildew (*Podosphaera clandestina*)
C-035	Cherry	Healthy
C-036	Soybean	Frogeye Leaf Spot (*Cercospora sojina*)
C-037	Soybean	Healthy
C-038	Squash	Powdery Mildew (*Podosphaera xanthii*)
C-039	Blueberry	Healthy
C-040	Raspberry	Healthy
C-041	Rice	Bacterial Blight (*Xanthomonas oryzae*)
C-042	Rice	Blast (*Magnaporthe oryzae*)
C-043	Rice	Brown Spot (*Cochliobolus miyabeanus*)
C-044	Rice	Tungro Virus
C-045	Rice	Healthy
C-046	Cotton	Bacterial Blight (*Xanthomonas citri subsp. malvacearum*)
C-047	Cotton	Leaf Curl Virus
C-048	Cotton	Target Spot
C-049	Cotton	Healthy
C-050	Mango	Anthracnose (*Colletotrichum gloeosporioides*)
C-051	Mango	Bacterial Canker (*Xanthomonas campestris*)

Table A.3. Complete 102-Class Label Taxonomy (Part 3 of 4)

Class ID	Plant Species	Disease / Condition
C-052	Mango	Gall Midge
C-053	Mango	Powdery Mildew (*Oidium mangiferae*)
C-054	Mango	Sooty Mould
C-055	Mango	Healthy
C-056	Eggplant	Cercospora Leaf Spot
C-057	Eggplant	Phomopsis Blight (*Phomopsis vexans*)
C-058	Eggplant	Phytophthora Blight
C-059	Eggplant	Healthy
C-060	Cucumber	Downy Mildew (*Pseudoperonospora cubensis*)
C-061	Cucumber	Powdery Mildew (*Sphaerotheca fuliginea*)
C-062	Cucumber	Target Spot
C-063	Cucumber	Healthy
C-064	Cauliflower	Bacterial Spot
C-065	Cauliflower	Black Rot (*Xanthomonas campestris*)
C-066	Cauliflower	Downy Mildew
C-067	Cauliflower	Healthy
C-068	Jute	Stem Rot (*Macrophomina phaseolina*)
C-069	Jute	Healthy
C-070	Wheat	Leaf Rust (*Puccinia triticina*)
C-071	Wheat	Stem Rust (*Puccinia graminis*)
C-072	Wheat	Yellow Rust (Stripe Rust, *Puccinia striiformis*)
C-073	Wheat	Powdery Mildew (*Blumeria graminis*)
C-074	Wheat	Healthy
C-075	Garlic	Downy Mildew (*Peronospora destructor*)
C-076	Garlic	Healthy
C-077	Onion	Botrytis Leaf Blight (*Botrytis squamosa*)

Table A.4. Complete 102-Class Label Taxonomy (Part 4 of 4)

Class ID	Plant Species	Disease / Condition
C-078	Onion	Purple Blotch (*Alternaria porri*)
C-079	Onion	Healthy
C-080	Lemon	Canker (*Xanthomonas axonopodis*)
C-081	Lemon	Greening (Huanglongbing)
C-082	Lemon	Healthy
C-083	Pomegranate	Anthracnose (*Colletotrichum gloeosporioides*)
C-084	Pomegranate	Bacterial Blight (*Xanthomonas axonopodis*)
C-085	Pomegranate	Healthy
C-086	Banana	Black Sigatoka (*Mycosphaerella fijiensis*)
C-087	Banana	Panama Disease (*Fusarium oxysporum*)
C-088	Banana	Healthy
C-089	Sugarcane	Red Rot (*Colletotrichum falcatum*)
C-090	Sugarcane	Rust (*Puccinia melanocephala*)
C-091	Sugarcane	Yellow Leaf Disease
C-092	Sugarcane	Healthy
C-093	Guava	Anthracnose
C-094	Guava	Fruit Canker
C-095	Guava	Healthy
C-096	Papaya	Ring Spot Virus
C-097	Papaya	Powdery Mildew
C-098	Papaya	Healthy
C-099	Cassava	Brown Streak Disease
C-100	Cassava	Mosaic Disease
C-101	Cassava	Healthy
C-102	General	Unknown / Out-of-Distribution (flagged for expert review)

A.2 Dataset Class Distribution

Figure A.1 illustrates the class frequency distribution across the combined training corpus. Class imbalance was addressed through stratified sampling during dataset splitting and through class-weighted loss computation during training.

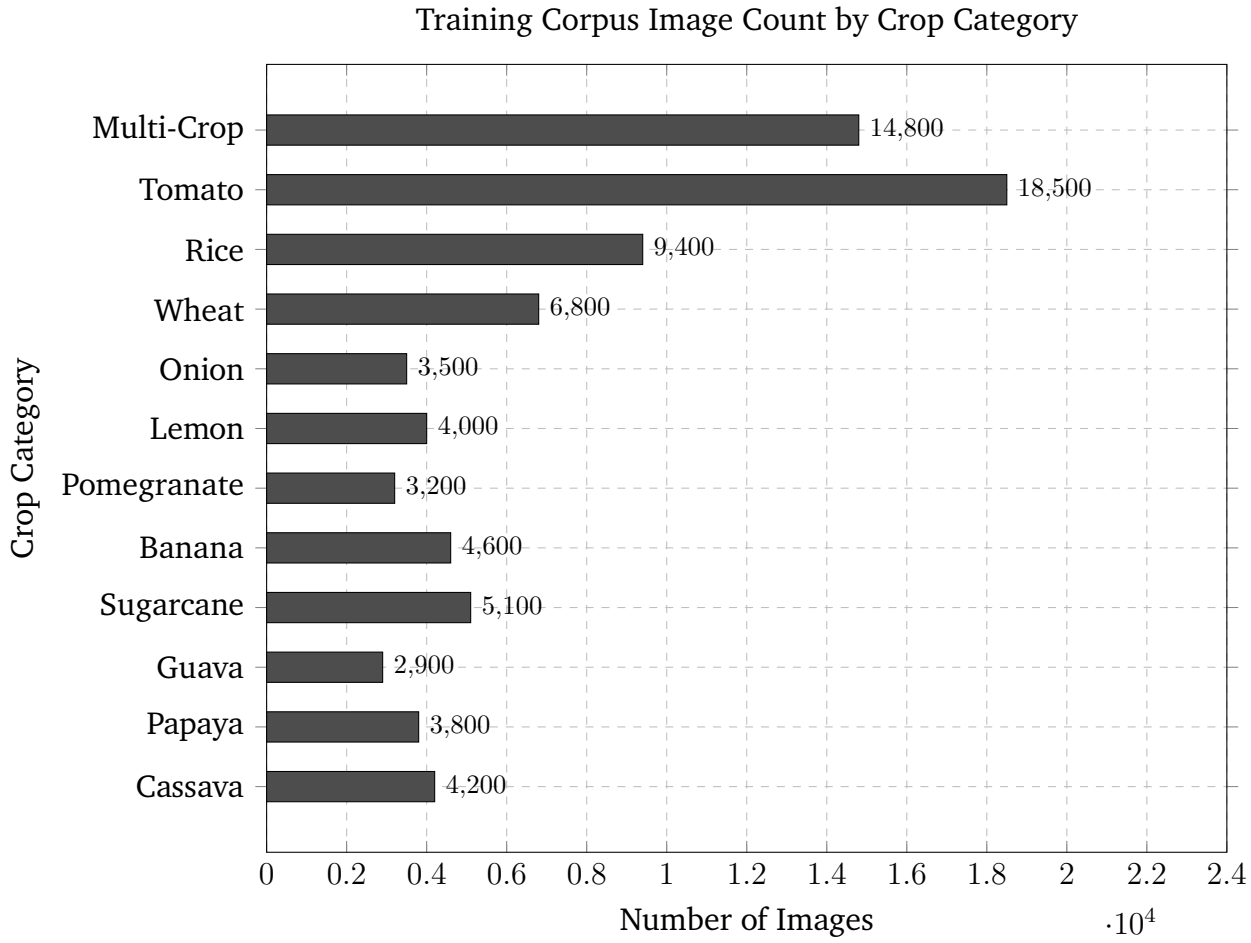

Figure A.1. Distribution of Images Across Crop Categories in the Combined Training Corpus

APPENDIX B

Project Visual Evidence Placeholders

This appendix records the remaining visual artefacts that should be exported from the implemented system before final submission.

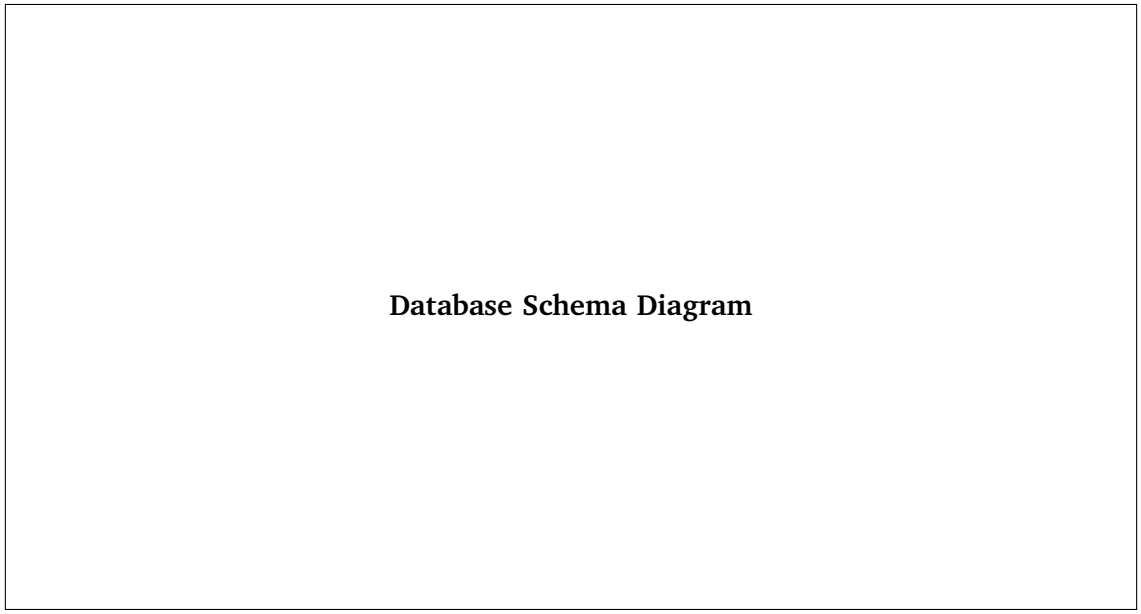

Figure B.1. Database Schema Diagram

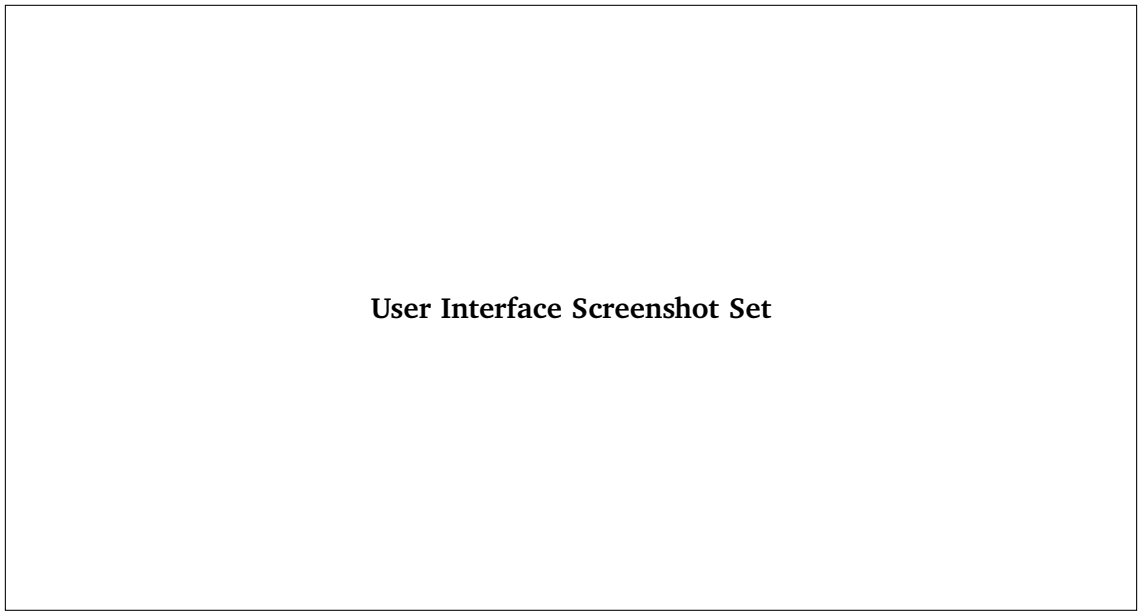

Figure B.2. User Interface Screenshot Set

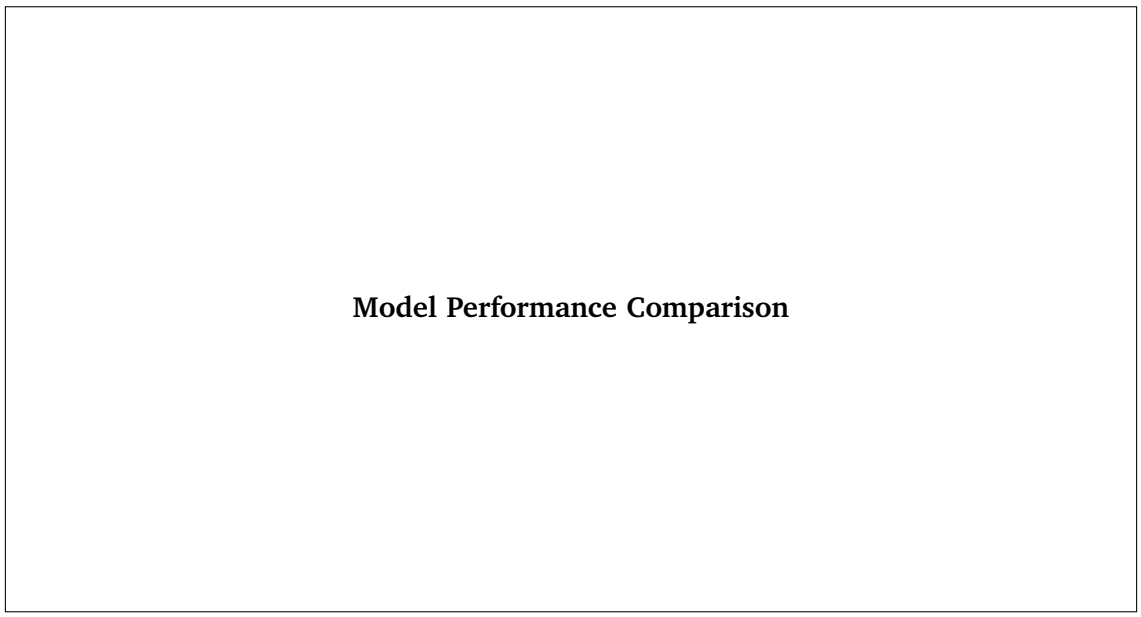

Figure B.3. Model Performance Comparison

Bibliography

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.

[2] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat, and N. Batra, "PlantDoc: A dataset for visual plant disease detection," pp. 249–253, 2020.

[3] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8." Ultralytics, 2023. Accessed: 2024-10-01.

[4] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pp. 234–241, Springer, 2015.

[5] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 618–626, 2017.

[6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[7] S. Ramírez, "FastAPI: Modern, fast (high-performance), web framework for building apis with Python." Tiangolo, 2023. Accessed: 2024-09-01.

Glossary

Artificial Intelligence The simulation of human cognitive processes—including reasoning, learning, and self-correction—by computer systems. In the context of NABTA, AI encompasses the deep learning models and language model components that drive automated plant disease diagnosis and consultation

Convolutional Neural Network A class of deep learning architecture specifically designed for processing grid-structured data such as images. CNNs employ learnable spatial filters to extract hierarchical feature representations from input images

Data Augmentation A set of image transformation techniques applied during model training to artificially increase dataset diversity and reduce overfitting. NABTA applies rotations, flips, colour jitter, and random erasing among other augmentation operations

Deep Learning A subfield of machine learning based on artificial neural networks with multiple representation-learning layers. Deep learning is the core enabling technology for the convolutional and segmentation models deployed in the NABTA pipeline

Explainable Artificial Intelligence A collection of methods and techniques that enable the outputs of AI systems to be understood and interpreted by human observers. NABTA employs Grad-CAM as its primary explainability mechanism

Inference The process of applying a trained machine learning model to new input data to generate predictions. In NABTA, inference refers to the sequential execution of the YOLOv8, U-Net, and CNN models on a submitted leaf image

Object Detection A computer vision task that simultaneously localises and classifies objects within an image, typically by producing bounding boxes accompanied by class labels and confidence scores

Pathogen A biological agent capable of causing disease in a host organism. In an agricultural context, plant pathogens include fungi, bacteria, viruses, and parasitic organisms responsible for crop diseases

Precision A classification metric defined as the ratio of true positive predictions to all positive predictions made by the model: Precision $= \text{TP}/(\text{TP} + \text{FP})$

Recall A classification metric defined as the ratio of true positive predictions to all actual positive instances in the dataset: Recall $= \text{TP}/(\text{TP} + \text{FN})$

Region of Interest A spatially bounded sub-region of an image selected for further analysis. In the NABTA pipeline, the YOLOv8 detection model identifies the infected leaf region as the region of interest for downstream processing

Semantic Segmentation A computer vision task in which every pixel of an input image is assigned a class label. In NABTA, semantic segmentation is performed by the U-Net model to isolate the leaf foreground from background content

System Usability Scale A standardised ten-item questionnaire used to assess the perceived usability of a software system. Scores range from 0 to 100, with scores above 80.3 classified as Excellent

Transfer Learning A machine learning technique in which a model pre-trained on a large dataset is adapted to a related task with a smaller domain-specific dataset. The U-Net in NABTA uses an EfficientNet-B0 backbone pre-trained on ImageNet

Vector Database A specialised database designed to store and retrieve large numerical vectors efficiently, typically using approximate nearest-neighbour algorithms. NABTA uses a vector database to index knowledge base embeddings for the RAG consultation module